

FC7300F8MDQ Errata Sheet

Rev.0.5

Table of Contents

Table of Contents	2
Chapter 1 Introduction	4
1.1 Conventions	4
1.2 Summary	4
1.3 Arm-related Errata	5
1.4 Errata ID Mapping (Previous vs. Current)	5
Chapter 2 Device Errata	7
2.1 ERR_ADC_3612112	7
2.2 ERR_ADC_3612113	7
2.3 ERR_Debug_3514113	9
2.4 ERR_Debug_3514112	10
2.5 ERR_DMA_3613112	10
2.6 ERR_DMA_3614121	13
2.7 ERR_FLEXCAN_3519117	15
2.8 ERR_FLEXCAN_3519113	16
2.9 ERR_LU_3617113	17
2.10 ERR_HSADC_3712112	18
2.11 ERR_QDT_3612114	19
2.12 ERR_RGM_3522113	21
2.13 ERR_RGM_3614122	21
2.14 ERR_RGM_3615114	22
2.15 ERR_SCG_3712113	22
2.16 ERR_SCG_3715113	23
2.17 ERR_SSI_3612116	24
2.18 ENET Errata	25
2.18.1 Synopsys ID: 4171346	25
2.18.2 Synopsys ID: 3900281	25
2.18.3 Synopsys ID: 3876182	27

2.18.4	Synopsys ID: 3873623	28
2.18.5	Synopsys ID: 3683170	29
2.18.6	Synopsys ID: 3413311	30
2.18.7	Synopsys ID: 3380358	32
2.18.8	Synopsys ID: 3379533	33
2.18.9	Synopsys ID: 3368939	34
2.18.10	Synopsys ID: 3340890	35
2.18.11	Synopsys ID: 3338201	37
2.18.12	Synopsys ID: 3309847	38
2.18.13	Synopsys ID: 3294970	39
2.18.14	Synopsys ID: 3290261	40
2.18.15	Synopsys ID: 3290231	41
2.18.16	Synopsys ID: 3288309	42
2.18.17	Synopsys ID: 6129927	43
2.18.18	Synopsys ID: 3275809	44
2.18.19	Synopsys ID: 4974260	45
2.18.20	Synopsys ID: 5121311	46
2.18.21	Synopsys ID: 5584644	47
2.18.22	Synopsys ID: 5575379	49
2.18.23	Synopsys ID: 5938448	51
2.18.24	Synopsys ID: 3542857	52
Revision History.....		54

Chapter 1 Introduction

This document describes the deviations of real behavior of the Flagchip FC7300F8MDQ series microcontroller (MCU) from the current documentation.

1.1 Conventions

Each erratum ID follows the pattern: **ERR_[Module Name]_[Number]**:

- **ERR:** Errata
- **Module Name:** The name of affected peripherals or subsystems
- **Number:** A seven-digit number indicating the sequence of an erratum

1.2 Summary

The following table summarizes the identified device limitations for quick reference.

Function	Erratum ID	Description	Status*
ADC	ERR_ADC_3612112	Setting SCG_SIRCCSR.LPWKE to 0 when SIRC is configured to be always on in Standby mode will result in the ADC being unable to be triggered.	A
	ERR_ADC_3612113	For a level-triggered situation, the captured timestamp cannot accurately represent the trigger time of ADC, PTIMER or SDADC because the capture behavior persists when the trigger is at a high level.	A
Debug	ERR_Debug_3514113	When the user switches from JTAG to SWD for programming or debugging, if PTA10 is configured as ALT7, the chip will drive PTA10 low. If there is an external high-level drive on PTA10 at this time, it may cause a drive conflict.	A
	ERR_Debug_3514112	In the debug mode, a lockstep error may happen after reset if certain conditions are met.	A
DMA	ERR_DMA_3613112	A DMAMUX slot available with DMA requests from two different modules cannot be assigned with DMA0 and DMA1 requests from different modules simultaneously.	A
	ERR_DMA_3614121	The DMA Monitor may incorrectly set error flags in DMA_MON_ERR_STATUS when users attempt to read DMA CFG registers during DMA transfers.	A
FLEXCAN	ERR_FLEXCAN_3519117	Enhanced Rx FIFO may lose a frame when a message buffer is locked or C/S write occurs.	A
	ERR_FLEXCAN_3519113	Cannot use MB0-MB7 for reception when Enhanced Rx FIFO is enabled.	A
LU	ERR_LU_3617113	If the Data Unit in the LU module is configured to flexible mode with the continuous dual-edge receive/transmit function enabled, and both DMA and interrupts are activated simultaneously, the LU fails to generate an interrupt to the CPU after the last 32-bit data is transmitted/received, resulting in an error in data updates.	A
HSADC	ERR_HSADC_3712112	Under specific conditions, the HSADC may produce one or two erroneous conversion results at the beginning of a normal ADC conversion sequence.	A

Function	Erratum ID	Description	Status*
QDT	ERR_QDT_3612114	The width of LECNT and POSDCNT counters is 16-bit, which may not be enough to record the time between Phase_A and Phase_B rising edges for speed calculation.	A
RGM	ERR_RGM_3522113	The chip may enter the reset state repeatedly for a long period if the RESET_B AON_CLK filter function is enabled with a certain external RC filter value.	A
	ERR_RGM_3614122	If a CPUx (except CPU0)-related reset source is configured as a core-only reset and the delay reset feature is enabled, unexpected bus errors may occur if CPUx does not enter a while(1) loop when the core reset is asserted.	A
	ERR_RGM_3615114	If a CPU1 reset is configured as a core-only reset, the AXBS bus will experience an additional halt of approximately 140 slow_clk cycles during CPU1 reset assertion.	A
SCG	ERR_SCG_3712113	If the PSTDIV1 bit field in either the PLL0_CFG or PLL1_CFG register is set to 1, there is a very low probability that the corresponding PLL (PLL0_CLK1 or PLL1_CLK1) will malfunction when it is enabled.	A
	ERR_SCG_3715113	Potential PLL lock failure due to asynchronous reset in pre-divider Logic.	A
SSI	ERR_SSI_3612116	For AK protocol SSI sensors, unexpected glitches cannot be completely filtered out due to the filter's limited bit width.	A
ENET	ENET Errata	ENET-related errata excerpted from Synopsys documents. Their Synopsys ID are 4171346, 3900281, 3876182, 3873623, 3683170, 3413311, 3380358, 3379533, 3368939, 3340890, 3338201, 3309847, 3294970, 3290261, 3290231, 3288309, 6129927, 3275809, 4974260, 5121311, 5584644, 5575379, 5938448, and 3542857.	A

NOTE: In the Status column, "A" indicates "Workaround Available", and "N" indicates "Workaround Not Available."

1.3 Arm-related Errata

For Arm-related errata, see attachments.

1.4 Errata ID Mapping (Previous vs. Current)

To ensure consistency across different chip variants, all errata IDs have been updated starting from Revision 0.4 of the errata sheet.

The following table provides a cross-reference between previous errata IDs and the current IDs. Customers using older versions of this document may refer to this table to locate corresponding errata items.

Table 1. Errata ID Mapping

Previous Errata ID	Current Errata ID
ERR_ADC_001	ERR_ADC_3612112
ERR_ADC_002	ERR_ADC_3612113
ERR_Debug_001	ERR_Debug_3514113
ERR_Debug_002	ERR_Debug_3514112

FC7300F8MDQ Errata Sheet

ERR_DMA_001	ERR_DMA_3613112
ERR_DMA_002	ERR_DMA_3614121
ERR_FLEXCAN_001	ERR_FLEXCAN_3519117
ERR_FLEXCAN_002	ERR_FLEXCAN_3519113
ERR_LU_001	ERR_LU_3617113
ERR_HSADC_001	ERR_HSADC_3712112
ERR_QDT_001	ERR_QDT_3612114
ERR_RGM_001	ERR_RGM_3522113
ERR_RGM_002	ERR_RGM_3614122
ERR_RGM_003	ERR_RGM_3615114
ERR_SCG_001	ERR_SCG_3712113
ERR_SSI_001	ERR_SSI_3612116

Chapter 2 Device Errata

This section provides a detailed description and workarounds (if available) of all errata.

2.1 ERR_ADC_3612112

Description

EN	In the ADC Sequence Group Compare standby mode application, when the SIRC is not configured to be always on in Standby mode, setting the SCG_SIRCCSR.LPWKE bit enables the SIRC for the ADC functional clock upon receiving a trigger. The SIRC is automatically disabled after the ADC conversion and compare operations are complete. According to the initial design, if the SIRC is configured to be always on in Standby mode, the SIRCCSR[LPWKE] bit can be set to 0. However, a defect exists where setting SIRCCSR[LPWKE] to 0 when SIRC is configured to be always on in Standby mode will result in the ADC being unable to be triggered.
CN	在 ADC 序列组比较待机模式应用中，当 SIRC 未配置为在待机模式（Standby mode）下常开时，设置 SCG_SIRCCSR.LPWKE 位可在接收到触发信号时为 ADC 功能时钟使能 SIRC。在 ADC 转换和比较操作完成后，SIRC 将自动关闭。 按初始设计，若 SIRC 已配置为在待机模式下常开，则 SIRCCSR[LPWKE] 位可设置为 0。但存在一个问题：当 SIRC 配置为在待机模式下常开时，如果将 SIRCCSR[LPWKE] 设为 0，会导致 ADC 无法被触发。

Workaround

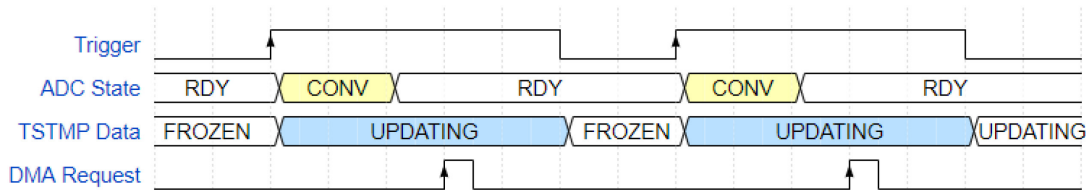
EN	To prevent the ADC from failing to capture the trigger for Sequence Group Compare applications, the SIRCCSR[LPWKE] bit should be set to 1, regardless of whether the SIRC is configured to be always on or not in Standby mode.
CN	为确保序列组比较应用中的 ADC 能正常捕获触发信号，无论 SIRC 是否配置为待机模式下常开，都应将 SIRCCSR[LPWKE] 位设置为 1。

2.2 ERR_ADC_3612113

Description

EN	There are 4 registers: SCM_TSTMP_RES0, SCM_TSTMP_RES1, SCM_TSTMP_RES2, and SCM_TSTMP_RES3, which are used to store the timestamps captured by triggers from ADC, PTIMER, or SDADC. The timestamps are intended to record the assertion time of the trigger, and theoretically, the capture behavior for each trigger should occur only once. However, there is a defect where the capture behavior persists when the trigger is at a high level. Therefore, for a level-triggered situation, the captured timestamp indicates the falling edge time of the trigger instead of the rising edge, which results in the timestamp being unable to accurately represent the trigger time of ADC, PTIMER, or SDADC. Furthermore, when the ADC conversion is completed and the level-trigger remains high, if there is a DMA request to transfer register data in sequential addresses, such as SCM_TSTMP_RES0, SGMP_ADC_GROUP0_RES0, and
----	--

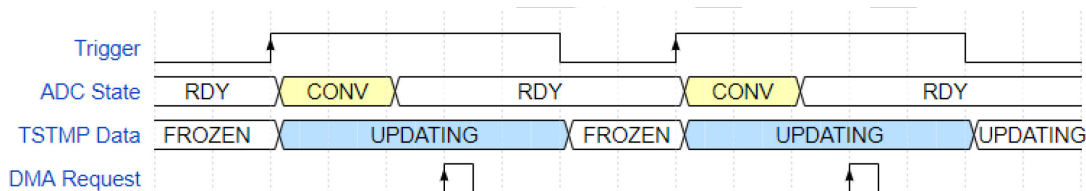
SCM_ADC_GROUP0_RES1, the timestamp stored in the transferred **SCM_TSTMP_RES0** only represents the assertion time of the DMA request and has nothing to do with the assertion or de-assertion time of the trigger.



It should be noted that pulse-based triggers with a width approximately equal to 1 bus clock cycle will not encounter this issue, because the brief high-level duration allows for only a single capture event.

CN 共有 4 个寄存器：**SCM_TSTMP_RES0**、**SCM_TSTMP_RES1**、**SCM_TSTMP_RES2** 和 **SCM_TSTMP_RES3**，用于存储由 ADC、PTIMER 或 SDADC 触发信号捕获的时间戳。这些时间戳本应记录触发信号的上升沿时间，理论上每个触发信号仅应触发一次捕获行为。然而，存在一个问题：当触发信号保持高电平时，捕获行为会持续发生。因此，在电平触发场景下，实际捕获的时间戳对应的是触发信号的下降沿时间，而非上升沿，导致时间戳无法准确表示 ADC、PTIMER 或 SDADC 的真实触发时刻。

此外，如果 ADC 转换完成时电平触发信号仍保持高电平，且 DMA 请求以连续地址传输寄存器数据（例如依次传输 **SCM_TSTMP_RES0**、**SGMP_ADC_GROUP0_RES0**、**SCM_ADC_GROUP0_RES1**），则传输的 **SCM_TSTMP_RES** 中的时间戳仅表示 DMA 请求的发起时间，与触发信号的断言/解除时间无关。



需注意，宽度约等于 1 个总线时钟周期的脉冲触发不受此问题。由于高电平持续时间极短，系统仅能执行单次捕获。

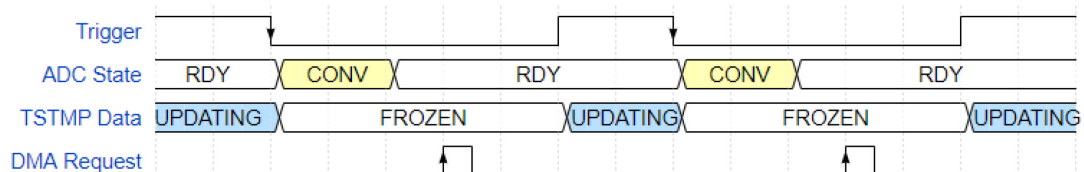
Workaround

EN For ADC and PTIMER:

To avoid the issue that the captured timestamp does not represent the trigger assertion time, the following workaround can be adopted:

- 1) Set **ADC_CFG1.TRIGMODE** to 010b to enable the ADC to use the falling edge trigger. Note that ADC discontinuous mode 1 and sequence group mode do not support the falling edge trigger.
- 2) If the PTIMER is expected to be triggered simultaneously with ADC (for timing control or DMA request generation), LU can be used to invert the original trigger via TRGSEL before sending it to PTIMER.
- 3) If the DMA is expected to transfer the timestamp in **SCM_TSTMP_RES0** or **SCM_TSTMP_RES1**, the request must be generated before the rising edge of the trigger.

Since the timestamp capture behavior stops at the falling edge of the trigger, the stored timestamp can reflect the trigger time of ADC.



For SDADC:

SCM_TSTMP_RES2 and **SCM_TSTMP_RES3** capture/lock may have this issue only when **TSTMP_LOCK[SDADCx_TMR_SRC_SEL]** in SCM is configured with Channel n (n = 0, 1, 2...) DMA request selected and the DMA request is held for some time (i.e., the DMA request is not processed in time).

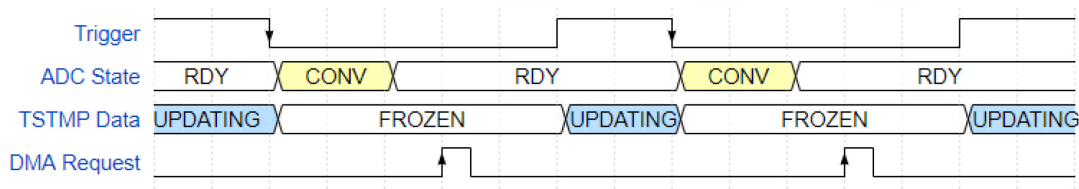
If an application cannot make sure if an DMA request can be processed in time, it is suggested to configure **TSTMP_LOCK[SDADCx_TMR_SRC_SEL]** in SCM with Channel n (n = 0, 1, 2...) conversion complete selected, the conversion complete event is 1 bus clock cycle trigger for the timestamp capture.

CN 对于 ADC 和 PTIMER:

为避免捕获的时间戳无法正确反映触发信号的断言时刻，可采用以下方法：

- 1) 将 **ADC_CFG1.TRIGMODE** 设置为 **010b**，使 ADC 仅在下降沿触发。**注意：**ADC 的非连续模式 1 (discontinuous mode 1) 和序列组模式 (sequence group mode) 不支持下降沿触发。
- 2) 如需 PTIMER 与 ADC 同步触发（用于时序控制或 DMA 请求生成），可通过 TRGSEL 选择原始触发信号，并利用 LU 将其反相后再输入 PTIMER。
- 3) 如需 DMA 传输 **SCM_TSTMP_RES0** 或 **SCM_TSTMP_RES1** 中的时间戳，必须在触发信号上升沿之前生成 DMA 请求。

由于时间戳捕获行为在触发信号下降沿终止，存储的时间戳可正确反映 ADC 的触发时刻。

**对于 SDADC:**

仅当满足以下条件时，**SCM_TSTMP_RES2** 和 **SCM_TSTMP_RES3** 的捕获/锁定可能出现时间戳错误：

- SCM 模块中的 **TSTMP_LOCK[SDADCx_TMR_SRC_SEL]** 配置为通道 n (n = 0, 1, 2...) 的 DMA 请求 (Channel n DMA request selected) 触发，且该 DMA 请求因未被及时处理而持续保持高电平。

若应用无法确保 DMA 请求能被及时处理，建议将 **TSTMP_LOCK[SDADCx_TMR_SRC_SEL]** 配置为通道 n (n = 0, 1, 2...) 的转换完成事件 (Channel n conversion complete selected) 触发。该转换完成事件为 1 个总线时钟周期的脉冲触发，可避免电平持续问题，确保时间戳准确捕获。

2.3 ERR_Debug_3514113**Description**

EN	When the user switches from JTAG to SWD for programming or debugging, if PTA10 is configured as ALT7, the chip will drive PTA10 low. If there is an external high-level drive on PTA10 at this time, it may cause a drive conflict.
CN	当用户从 JTAG 切换到 SWD 进行烧录或者调试时，如果 PTA10 选择 ALT7，芯片会给 PTA10 驱动低电平。如果此时 PTA10 还有外部高电平驱动，会导致驱动冲突风险。

Workaround:

EN	Before switching to SWD mode, if PTA10 is configured as ALT7, ensure that the PTA10 has no external high-level drive to avoid conflicts.
CN	在切换成 SWD 进行烧录或者调试时，如果 PTA10 选择 ALT7，需要确保 PTA10 的 pad 没有外部高电平驱动冲突。

2.4 ERR_Debug_3514112**Description**

EN	In the debug mode, a lockstep error may happen after reset if the below conditions are all meet: 1) CORE is halted; 2) DWT is enabled; 3) DWT_CYCCNT is active when a reset happens; 4) Read DWT_CYCCNT after the reset finished. The root cause of this erratum is the DWT_CYCCNT mismatch.
CN	在调试模式（Debug mode）下，如果同时满足以下所有条件，系统复位后可能出现锁步错误： 1) 内核处于暂停状态； 2) 数据观察点与跟踪单元（DWT）已使能； 3) 复位发生时 DWT_CYCCNT（周期计数器）正在运行； 4) 复位完成后读取了 DWT_CYCCNT 值。 此问题的根本原因在于 DWT_CYCCNT 数值不匹配。

Workaround

EN	There are two workarounds available: • Workaround 1: After the chip is halted, disable DWT before reset. • Workaround 2: Disable DWT_CYCCNT on the startup routine in the debug mode.
CN	有以下两种规避方案： • 规避方案 1： 芯片暂停后，在复位前禁用 DWT 功能。 • 规避方案 2： 在调试模式的启动例程中禁用 DWT_CYCCNT。

2.5 ERR_DMA_3613112**Description**

EN	The DMAMUX slots listed in the following table are available with DMA requests from different modules. However, due to a design constraint, such a DMAMUX slot cannot be assigned with DMA0 and DMA1 requests from different modules simultaneously.
-----------	--

CN 下表所列的 DMAMUX 通道可用于接收来自不同模块的 DMA 请求。但由于设计限制，此类 DMAMUX 通道无法同时分配来自不同模块的 DMA0 和 DMA1 请求。

Slot_Num	Module	DMA_Request Description	Instance
63	FCUART	FCUART Receive DMA Request	FCUART11
	LU	LU DMA Request	LU0
64	FCUART	FCUART Transmit DMA Request	FCUART11
	LU	LU1 DMA Request	LU1
65	FCUART	FCUART Receive DMA Request	FCUART12
	TPU	TPU CH8-11 DMA Request	TPU0
66	FCUART	FCUART Transmit DMA Request	FCUART12
	TPU	TPU CH12-15 DMA Request	TPU0
67	FCUART	FCUART Receive DMA Request	FCUART13
	TPU	TPU CH16-19 DMA Request	TPU0
68	FCUART	FCUART Transmit DMA Request	FCUART13
	TPU	TPU CH20-23 DMA Request	TPU0
77	Port Control	PORTA DMA Request	PORTA
	TPU	TPU CH0-3 DMA Request	TPU0
78	Port Control	PORTB DMA Request	PORTB
	TPU	TPU CH4-7 DMA Request	TPU0
79	Port Control	PORTC DMA Request	PORTC
	eFTU	eFTU DMA SRC1 Request	eFTU1
80	Port Control	PORTD DMA Request	PORTD
	eFTU	eFTU DMA SRC2 Request	eFTU1
81	Port Control	PORTE DMA Request	PORTE
	eFTU	eFTU DMA SRC4 Request	eFTU1
82	Port Control	PORTF DMA Request	PORTF
	eFTU	eFTU DMA SRC5 Request	eFTU1
83	Port Control	PORTG DMA Request	PORTG
	eFTU	eFTU DMA SRC1 Request	eFTU2
84	Port Control	PORTH DMA Request	PORTH
	eFTU	eFTU DMA SRC2 Request	eFTU2
85	Port Control	PORTI DMA Request	PORTI
	eFTU	eFTU DMA SRC4 Request	eFTU2
86	AONTIMER	AONTIMER DMA Request	AONTIMER0
	eFTU	eFTU DMA SRC5 Request	eFTU2
	TPU	TPU CH24 DMA Request	TPU0
87	CMP	CMP DMA Request	CMP0
	eFTU	eFTU DMA SRC0 Request	eFTU0
	TPU	TPU CH25 DMA Request	TPU0
90	TPU	TPU CH26 DMA Request	TPU0
	eFTU	eFTU DMA SRC3 Request	eFTU0
	PTIMER	PTIMER DMA Request	PTIMER0

91	eFTU	eFTU DMA SRC0 Request	eFTU1
	TPU	TPU CH27 DMA Request	TPU0
	PTIMER	PTIMER DMA Request	PTIMER1
92	eFTU	eFTU DMA SRC3 Request	eFTU1
	PTIMER	PTIMER DMA Request	PTIMER2
93	eFTU	eFTU DMA SRC0 Request	eFTU2
	PTIMER	PTIMER DMA Request	PTIMER3
95	eFTU	eFTU DMA SRC3 Request	eFTU2
	ENET PPS Timer	ENET Period Timer Trigger CH1	ENET
96	Flash	Flash Program DMA Add Request	Flash
	ENET PPS Timer	ENET Period Timer Trigger CH2	ENET
97	Flash	Flash Program DMA Data Request	Flash
	ENET PPS Timer	ENET Period Timer Trigger CH3	ENET
102	FTU	FTU All Odd Channels OR DMA Request	FTU4
	TPU	TPU CH28 DMA Request	TPU0
103	FTU	FTU All Channels OR DMA Request	FTU5
	TPU	TPU CH29 DMA Request	TPU0
104	FTU	FTU All Channels OR DMA Request	FTU6
	TPU	TPU CH30 DMA Request	TPU0
105	FTU	FTU All Channels OR DMA Request	FTU7
	TPU	TPU CH31 DMA Request	TPU0
106	FLEXCAN	FLEXCAN DMA Request	FLEXCAN10
	FTU	FTU Channel 0 DMA Request	FTU0
107	FLEXCAN	FLEXCAN DMA Request	FLEXCAN11
	FTU	FTU Channel 2 DMA Request	FTU0
108	FLEXCAN	FLEXCAN DMA Request	FLEXCAN12
	FTU	FTU Channel 4 DMA Request	FTU0
109	FLEXCAN	FLEXCAN DMA Request	FLEXCAN13
	FTU	FTU Channel 6 DMA Request	FTU0
118	SENT	SENT Channel 0 Slow DMA Request	SENT0
	FTU	FTU Channel 0 DMA Request	FTU3
119	SENT	SENT Channel 1 Slow DMA Request	SENT0
	FTU	FTU Channel 2 DMA Request	FTU3
120	SENT	SENT Channel 2 Slow DMA Request	SENT0
	FTU	FTU Channel 4 DMA Request	FTU3
121	SENT	SENT Channel 3 Slow DMA Request	SENT0
	FTU	FTU Channel 6 DMA Request	FTU3
122	SENT	SENT Channel 0 Slow DMA Request	SENT1
	FTU	FTU Channel 0 DMA Request	FTU4
123	SENT	SENT Channel 1 Slow DMA Request	SENT1
	FTU	FTU Channel 2 DMA Request	FTU4
124	SENT	SENT Channel 2 Slow DMA Request	SENT1

	125	FTU	FTU Channel 4 DMA Request	FTU4
		SENT	SENT Channel 3 Slow DMA request	SENT1
		FTU	FTU Channel 6 DMA request	FTU4

Workaround

EN	These DMAMUX slots must be assigned with DMA0 and DMA1 requests from the same module simultaneously.
CN	对于上述 DMAMUX 通道，必须同时分配来自同一模块的 DMA0 和 DMA1 请求。

2.6 ERR_DMA_3614121

Description

EN	The DMA Monitor may incorrectly set error flags in DMA_MON_ERR_STATUS when users attempt to read DMA CFG registers during DMA transfers. NOTE: Applicable to DMA1 only.
CN	当用户在 DMA 传输期间尝试读取 DMA 配置寄存器时，DMA 监控（DMA Monitor）功能可能会错误地置位寄存器 DMA_MON_ERR_STATUS 中的错误标志。 注意： 仅适用于 DMA1。

Workaround:

EN	There are three workarounds available: • Workaround 1: When DMA Monitor is enabled (MON_CHK_EN set): ■ Only perform write operations to the following DMA CFG registers: • DMA_CFG_SADDRn • DMA_CFG_SOFFn • DMA_CFG_ATTRn • DMA_CFG_DADDRn • DMA_CFG_DOFFn • DMA_CFG_NBYTES_ILNOn • DMA_CFG_NBYTES_ILOFFNOn • DMA_CFG_NBYTES_ILOFFYESn • DMA_CFG_CLC_CHTRGENNOn • DMA_CFG_CLC_CHTRGENYESn ■ No restrictions on the following DMA CFG registers (read/write allowed during transfer): • DMA_CFG_SLASTn
-----------	--

	• DMA_CFG_DLASTn • DMA_CFG_CSRn • DMA_CFG_BLC_CHTRGENNOn • DMA_CFG_BLC_CHTRGENYESn • Workaround 2: Disable the DMA Monitor (MON_CHK_EN clear) at the beginning of all DMA channel applications, and use software CRC to verify source and destination data later. • Workaround 3: If functional safety transactions related to the DMA Monitor are required, use DMA0 with Lockstep instead.
CN	有以下三种规避方案： • 规避方案 1： 当使能 DMA 监控功能（MON_CHK_EN 置位）时： ■ 仅允许写入以下 DMA 配置寄存器： • DMA_CFG_SADDRn • DMA_CFG_SOFFn • DMA_CFG_ATTRn • DMA_CFG_DADDRn • DMA_CFG_DOFFn • DMA_CFG_NBYTES_ILNOn • DMA_CFG_NBYTES_ILOFFNOn • DMA_CFG_NBYTES_ILOFFYESn • DMA_CFG_CLC_CHTRGENNOn • DMA_CFG_CLC_CHTRGENYESn ■ 对以下 DMA 配置寄存器无操作限制（传输期间允许读写）： • DMA_CFG_SLASTn • DMA_CFG_DLASTn • DMA_CFG_CSRn • DMA_CFG_BLC_CHTRGENNOn • DMA_CFG_BLC_CHTRGENYESn • 规避方案 2： 在所有 DMA 通道应用开始时禁用 DMA 监控功能（清除 MON_CHK_EN 位），后续通过软件 CRC 来校验源数据和目标数据。 • 规避方案 3： 如需使用 DMA 的功能安全机制来监测相关传输，请改用支持锁步模式（Lockstep）的 DMA0。

2.7 ERR_FLEXCAN_3519117

Description

EN

Enhanced Rx FIFO may lose a frame when a message buffer is locked or C/S write occurs.

An incoming frame that should be stored in the Enhanced Rx FIFO is instead dropped if either of the following conditions is true:

(a) The current value of the internal Enhanced Rx FIFO write pointer matches the index of an affected message buffer that is currently locked (see the figure below).

(b) The application writes to the C/S word of an affected message buffer in 1 special bus tick in Move-in process.

CN

增强型 Rx FIFO（Enhanced Rx FIFO）在消息缓冲区（MB）锁定或控制/状态字（C/S 字）写入时可能会丢帧。

当满足以下任一条件时，本应存入增强型 Rx FIFO 的接收帧会被丢弃：

(a) 增强型 Rx FIFO 内部写指针的当前值，与某个已锁定的受影响的 MB 索引相匹配（见下图）；

(b) FLEXCAN 在搬运过程（move-in process）中，在特定的 1 个总线时钟周期内向受影响的 MB C/S 字写入时。

Message Buffer (MB)		Enhanced Rx FIFO	FIFO Pointer
Affected MBs	MB 79		19
	MB 69		18
	MB 59		17
	MB 49		16
	MB 39		15
	MB 29		14
	MB 19		13
	MB 9		12
Unaffected MBs			11
			10
			9
			8
Affected MBs	MB 87		7
	MB 77		6
	MB 67		5
	MB 57		4
	MB 47		3
	MB 37		2
	MB 27		1
	MB 17		0

Workaround

EN	This issue can be avoided by using the affected MBs only with the following restrictions: - Do not use the affected MBs to transmit frames. - When using an affected MB as a receive MB, configure the MB once at initialization and do not write to the C/S word of that MB again after initialization. - When reading a message from a receive MB, lock the MB by reading the message buffer C/S word, read the data, and unlock the MB by reading the Free Running Timer, in one atomic operation. The time from locking
-----------	--

	the MB, reading the data, and unlocking the MB should be kept to the less than the time of one minimum-length CAN frame to ensure that a new message is not received while the MB is locked. Examples of minimum-length CAN frames are: • A Classic CAN frame with BASE ID = 0×199, RTR = 1, IDE = 0, FDF = 0, DLC = 0×7 (no stuff bits) • A CAN FD frame with BASE ID = 0×199, RTR/RRS = 0, IDE = 0, FDF = 1, BRS = 1, ESI = 1, DLC = 0×0 (only 6 fixed stuff bits) The issue can also be avoided by: • Not using the affected message buffers. • Not using the Enhanced Rx FIFO.
CN	使用受影响的 MB 时，遵循以下限制条件可规避此问题： • 禁止将受影响的 MB 用于发送帧。 • 如果将受影响的 MB 作为接收 MB，仅在初始化时配置一次，之后禁止再次写入其 C/S 字。 • 从接收 MB 读取消息时，必须通过原子操作完成以下操作： 1) 锁定 MB（读取其 C/S 字）； 2) 读取数据； 3) 解锁 MB（读取自由运行定时器值）。 所有操作耗时必须短于最短 CAN 帧的传输时间，以避免在 MB 锁定期间接收新消息。 最短 CAN 帧示例： • 经典 CAN 帧： BASE ID = 0×199, RTR = 1, IDE = 0, FDF = 0, DLC = 0×7（无填充位） • CAN FD 帧： BASE ID = 0×199, RTR/RRS = 0, IDE = 0, FDF = 1, BRS = 1, ESI = 1, DLC = 0×0（仅 6 个固定填充位） 其他规避方案： • 停用受影响的 MB。 • 停用增强型 Rx FIFO 功能。

2.8 ERR_FLEXCAN_3519113

Description

EN	Cannot use MB0 - MB7 for reception when Enhanced Rx FIFO is enabled. A message received into a receive message buffer may be lost without indication when the following sequence of conditions occurs: 1) The Enhanced Rx FIFO is enabled. 2) A message is received into any of MB0-MB7 and is not immediately read by the application.
----	--

	3) The application services one or more messages in the Enhanced Rx FIFO, thereby clearing the CAN_ERFSR.ERFDA bit one or more times. 4) Another message is received into a full and unserviced receive message buffer in the range MB0-MB7. In such a case, the CODE field of the affected message buffer should change from FULL to OVERRUN. However, depending on the message buffer number and the number of times CAN_ERFSR.ERFDA was cleared, the CODE field of the affected receive message buffer may remain FULL. Therefore, the application is unaware that a received message was overwritten and lost.
CN	当使能增强型 Rx FIFO（Enhanced Rx FIFO）时，MB0-MB7 无法用于接收消息。 以下条件依次发生时，接收消息缓冲区（MB）中的消息可能会无提示丢失： 1) 增强型 Rx FIFO 功能已使能； 2) 有消息接收至 MB0-MB7 中的任意缓冲区，但未被应用程序及时读取； 3) 应用程序处理增强型 Rx FIFO 中的一条或多条消息，导致 CAN_ERFSR.ERFDA 标志位被清除一次或多次； 4) 另一条消息被接收至 MB0-MB7 范围内已满但未及时读取的接收缓冲区。 此时，受影响缓冲区的 CODE 位域本应从 FULL 变为 OVERRUN。但若缓冲区编号和增强型 Rx FIFO 的索引相同，在 CAN_ERFSR.ERFDA 清除时，该位域可能保持为 FULL 状态，从而导致应用程序无法感知接收到的消息已被覆盖丢失。

Workaround

EN	Use either of the following workarounds to avoid this issue: • DO NOT use MB0-MB7 when the Enhanced Rx FIFO is enabled. • If you do use MB0-MB7 when the Enhanced Rx FIFO is enabled, be sure to service any pending received message buffer interrupts for MB0-MB7, including reading message data, before servicing messages received in the Enhanced Rx FIFO.
CN	可采用以下任一方式规避此问题： • 使能增强型 Rx FIFO 时，禁止使用 MB0-MB7； • 如果必须使能增强型 Rx FIFO 并使用 MB0-MB7，则必须优先处理 MB0-MB7 的已接收的消息缓冲区中断（包括读取消息数据），之后再处理增强型 Rx FIFO 中已接收的消息。

2.9 ERR_LU_3617113

Description

EN	If the Data Unit in the LU module is configured to flexible mode with the continuous dual-edge receive/transmit function enabled, and both DMA and interrupts are activated simultaneously, the LU fails to generate an interrupt to the CPU after the last 32-bit data is transmitted/received, resulting in an error in data updates.
CN	当用户将 LU 模块中数据单元（Data Unit）的工作模式配置为灵活模式，并开启连续双沿接收/发送功能，且同时使能 DMA 和中断时，LU 在接收/发送最后一组 32 位数据后无法产生中断给 CPU，从而导致数据更新出错。

Workaround

EN	If the Data Unit is configured to flexible mode with continuous dual-edge receive/transmit enabled, it is recommended to enable only DMA or interrupts (not both).
CN	如果数据单元被配置为灵活模式且开启了连续双沿接收/发送功能，建议仅使能 DMA 或仅使能中断（二者不可同时使能）。

2.10 ERR_HSADC_3712112

Description

EN	Under specific conditions, the HSADC may produce one or two erroneous conversion results at the beginning of a normal ADC conversion sequence. This issue occurs when the sampling operation starts after an idle period exceeding 15 μs. Specifically: - The HSADC is enabled but remains untriggered for longer than 15 μs. - More than 15 μs has elapsed since the start of the last sampling phase.
CN	在特定条件下，HSADC 模块在启动正常 ADC 转换序列时，其前 1-2 次转换结果可能出现错误。 该问题的触发条件是：采样操作启动前存在超过 15 μs 的空闲期。具体场景如下： - HSADC 已使能，但持续 15 μs 以上未被触发。 - 距上一次采样阶段启动的空闲间隔超过 15 μs。

Workaround

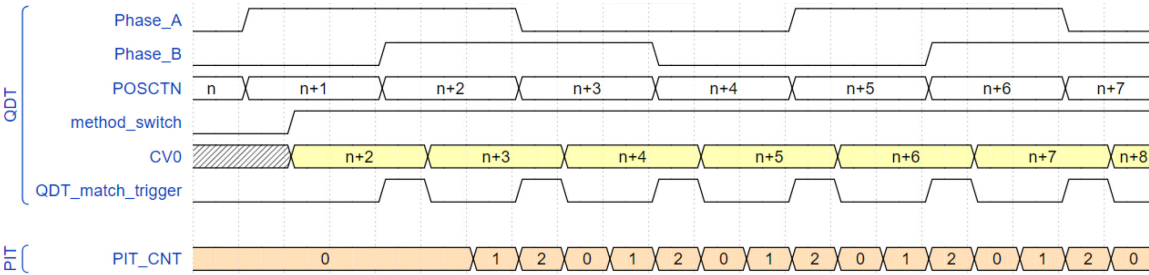
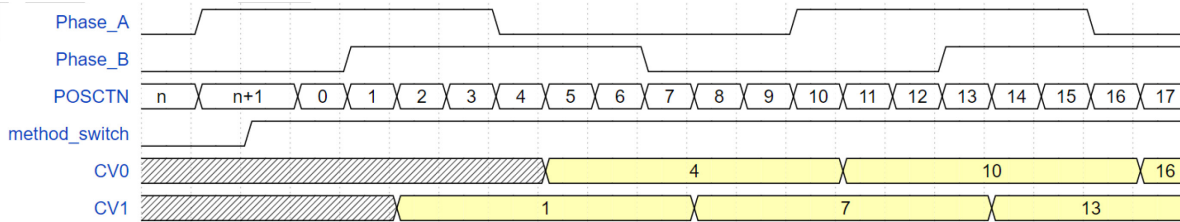
EN	To ensure data integrity, the following timing constraints must be met: Intra-Sequence Timing: If a single trigger initiates a sequence of multiple conversions, ensure that the time interval between the start of one sampling phase and the start of the next sampling phase is $< 15 \mu$s. This constraint is mandatory and cannot be bypassed. Inter-Trigger Timing: The idle time between two consecutive triggers must be $< 15 \mu$s. This interval is defined as the period from the start of sampling for the final conversion in the previous triggered sequence to the start of sampling for the first conversion in the current triggered sequence. If this interval is unavoidably longer than 15 μs, add two unused dummy conversions at the start of the expected sequence and discard their results.
CN	为确保数据完整性，必须满足以下时序约束： 序列内时序约束：若单个触发信号启动了一个包含多次转换的序列，则必须确保相邻两次采样的启动时间间隔 $< 15 \mu$s。此约束为强制性要求，无法规避。 触发间时序约束：两个连续触发信号之间的空闲时间必须 $< 15 \mu$s。此时间间隔定义为：从上一触发序列中末次转换的采样启动时刻，到当前触发序列中首次转换的采样启动时刻。若实际间隔无法满足 $< 15 \mu$s，请在目标转换序列的起始处添加两次无效的虚拟转换，并丢弃其转换结果。

2.11 ERR_QDT_3612114

Description

EN	The width of LECNT and POSDCNT counters is 16-bit, which may not be enough to record the time between Phase_A and Phase_B rising edges (tooth_period) for speed calculation.
CN	LECNT 与 POSDCNT 计数器的位宽为 16 位，可能无法完整记录 Phase_A 与 Phase_B 上升沿之间的时间间隔 (tooth_period) 以实现转速计算。

Workaround

EN	To avoid the issue that the width of LECTN and POSDCNT counters is not enough, there are two workarounds available: Workaround 1: If the tooth number is 1000, when the speed is lower than 120 RPM (configurable via software), and the method_switch is high, use PIT to record the tooth_period.  - Set CV0 = POSCNT + 1 to generate match_trigger to trigger a PIT enter interrupt. - Get tooth_period by reading the PIT counter in the PIT interrupt service routine. - Calculate the speed = 1/tooth_period. - Get the direction by reading the register QUADIR. NOTE: It is recommended not to reset POSCNT during speed measurement. Workaround 2: If the tooth number is 1000, when the speed is lower than 120 RPM (configurable via software), and the method_switch is high, switch QDT from the QUAD mode to the Input Capture (IC) mode to record the tooth_period.  - Switch QDT from the QUAD mode to the IC mode, and set channel 0/1 to capture rising and falling edges. - Enable the channel 0/1 interrupt. - Calculate the speed = 1/tooth_period: - In the channel 0 interrupt service routine, the tooth_period = (CV0-CV1). - In the channel 1 interrupt service routine, the tooth_period = (CV1-CV0).
----	--

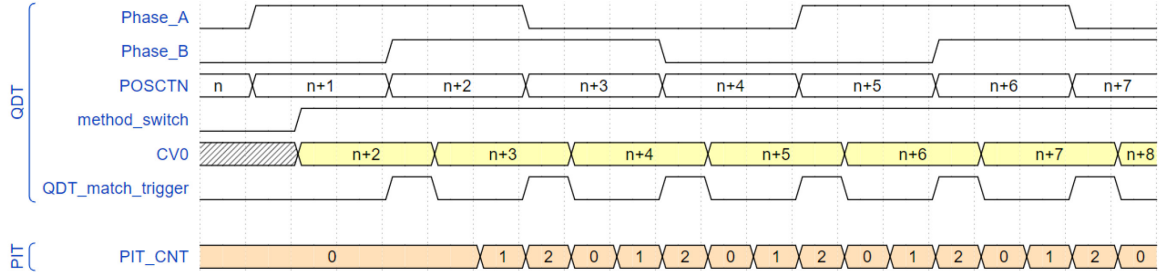
4) Get the direction by comparing the **POSCNT** values due to the exit of the QUAD mode.

NOTE: It is recommended not to reset **POSCNT** during speed measurement.

CN

为避免 **LECNT** 与 **POSDCNT** 计数器位宽不足的问题，可采用以下两种规避方案：

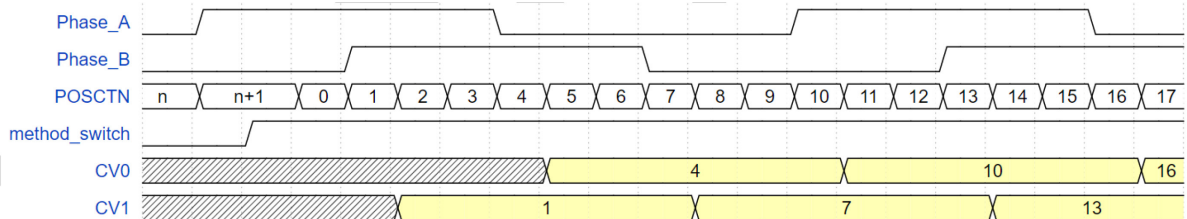
- **规避方案 1：**当齿数为 1000 且转速低于 120 RPM（可通过软件配置），且 **method_switch** 为高电平时，使用 PIT 模块记录 **tooth_period**。



- 1) 设置 **CV0 = POSCNT + 1**，生成 **match_trigger** 以触发 PIT 进入中断。
- 2) 在 PIT 中断服务程序中读取 PIT 计数器值获取 **tooth_period**。
- 3) 计算转速：转速 = $1/\text{tooth_period}$ 。
- 4) 通过读取寄存器 **QUADIR** 获取方向信息。

注意：建议在转速测量期间不要复位 **POSCNT**。

- **规避方案 2：**当齿数为 1000 且转速低于 120 RPM（可通过软件配置），且 **method_switch** 为高电平时，将 QDT 从正交编码模式（QUAD mode）切换为输入捕获模式（IC mode）以记录 **tooth_period**。



- 1) 将 QDT 从 QUAD 模式切换至 IC 模式，并设置通道 0/1 捕获上升沿与下降沿。
- 2) 启用通道 0/1 中断。
- 3) 计算转速：转速 = $1/\text{tooth_period}$ 。
 - 在通道 0 中断服务程序中， $\text{tooth_period} = (\text{CV0} - \text{CV1})$
 - 在通道 1 中断服务程序中， $\text{tooth_period} = (\text{CV1} - \text{CV0})$
- 4) 由于退出 QUAD 模式，需通过比较 **POSCNT** 值来获取方向信息。

注意：建议在转速测量期间不要复位 **POSCNT**。

2.12 ERR_RGM_3522113

Description:

EN	The chip may enter the reset state repeatedly for a long period if the RESET_B AON_CLK filter function is enabled with a certain external RC filter value. Condition: If the RESET_B pin AON_CLK filter function is enabled and the RC filter value on the RESET_B pin that causes RESET_B to rise from 0V to the VIH is larger than 2 AON_CLK periods, the system may be stuck at reset for a long time.
CN	如果在使能 RESET_B 引脚的 AON_CLK 滤波功能时，外部 RC 滤波参数设置不当，芯片可能会长时间反复进入复位状态。 触发条件： 当同时满足以下两点时，系统可能长时间卡在复位状态： 1) RESET_B 引脚的 AON_CLK 滤波功能已使能； 2) RESET_B 引脚的 RC 滤波值（使 RESET_B 从 0V 上升至 VIH 的时间）大于 2 个 AON_CLK 周期。

Workaround:

EN	There are two workarounds available: • Workaround 1: Do not use the AON_CLK filter, but use the external RC filter and bus filter functions instead. • Workaround 2: If using the AON_CLK filter, make sure the RC filter value that causes RESET_B to rise from 0 to VIH is less than 2 AON_CLK periods (for safety, $\leq 50 \mu s$ is recommended).
CN	有以下两种规避方案： • 规避方案 1： 不使用 AON_CLK 滤波功能，改用外部 RC 滤波结合总线滤波功能。 • 规避方案 2： 如果必须使用 AON_CLK 滤波功能，必须确保 RC 滤波值（使 RESET_B 从 0V 上升至 VIH 的时间）小于 2 个 AON_CLK 周期（建议保守取值 $\leq 50\mu s$）。

2.13 ERR_RGM_3614122

Description

EN	If a CPUx (except CPU0)-related reset source (such as LOCKUP, SYSRST, WDOG, INTM, and SWRST) is configured as a core-only reset and the delay reset feature is enabled, unexpected bus errors may occur if CPUx does not enter a while(1) loop when the core reset is asserted.
CN	如果与 CPUx（除 CPU0 外）相关的复位源（如 LOCKUP、SYSRST、WDOG、INTM 和 SWRST）被配置为仅内核复位（core-only reset），并且已使能延迟复位（delay reset）功能，那么当内核复位触发时，若 CPUx 未进入 while(1) 循环，则可能发生非预期的总线错误。

Workaround

EN	There are two workarounds available: • Workaround 1: For the same reset source, do not enable both the core-only reset and the delay reset feature simultaneously. • Workaround 2: If both the core-only reset and the delay reset feature are enabled, ensure that the corresponding CPUx will execute a while(1) loop when the core reset is asserted. It is recommended to place the while(1) code in TCM. By default, the RGM interrupt is sent to all CPUs, even if the reset is configured as core-only reset. It is suggested to route the interrupt only to the corresponding CPU by configuring SCM_INT_ROUTERn registers.
CN	有以下两种规避方案： • 规避方案 1： 对于同一复位源，不要同时使能“仅内核复位”和“延迟复位”功能。 • 规避方案 2： 如果必须同时使能“仅内核复位”和“延迟复位”功能，则需确保对应的 CPUx 在内核复位触发时执行 while(1) 循环。建议将该循环代码放置在 TCM 中。 默认情况下，即使复位被配置为“仅内核复位”，RGM 中断也会被发送至所有 CPU。建议通过配置寄存器 SCM_INT_ROUTERn，将该中断仅路由至对应的 CPU。

2.14 ERR_RGM_3615114

Description

EN	If a CPU1 reset (LOCKUP, SYSRST, WDOG, INTM, and SWRST) is configured as a core-only reset, the AXBS bus will experience an additional halt of approximately 140 slow_clk cycles during CPU1 reset assertion. During this period, all masters will be unable to access any memory or peripherals via the AXBS bus.
CN	如果将 CPU1 复位（包括 LOCKUP、SYSRST、WDG、INTM 和 SWRST）配置为仅内核复位（core-only reset），在 CPU1 复位生效期间，AXBS 总线将额外停滞约 140 个 slow_clk 周期。在此期间，所有主设备（masters）都将无法通过 AXBS 总线访问任何存储器或外设。

Workaround

EN	It is recommended to configure a CPU1 reset (LOCKUP, SYSRST, WDOG, INTM, and SWRST) as a system reset.
CN	建议将 CPU1 复位（LOCKUP、SYSRST、WDG、INTM 和 SWRST）配置为系统复位（system reset）。

2.15 ERR_SCG_3712113

Description

EN	If the PSTDIV1 bit field in either the PLL0_CFG or PLL1_CFG register is set to 1, there is a very low probability that the corresponding PLL (PLL0_CLK1 or PLL1_CLK1) will malfunction when it is enabled.
CN	当 PLL0CFG 或 PLL1CFG 寄存器中的 PSTDIV1 位域被配置为 1 时，开启对应的 PLL（PLL0_CLK1 或 PLL1_CLK1）后，存在极低概率导致该 PLL 工作异常。

Workaround

EN	Software must not configure the PSTDIV1 bit field to a value of 1 (which corresponds to a clock division ratio of 4) in either the PLL0CFG or PLL1CFG register. To achieve a clock division ratio of 4, configure the DIVH, DIVM, or DIVL bit fields in the corresponding PLL0DIV or PLL1DIV register instead.
CN	软件必须避免将 PLL0CFG 或 PLL1CFG 寄存器中的 PSTDIV1 位域配置为 1（即时钟分频比 = 4）。 如需将时钟分频比设置为 4，可以通过 PLL0DIV 或 PLL1DIV 寄存器中的 DIVH、DIVM 或 DIVL 位域进行配置。

2.16 ERR_SCG_3715113

Description

EN	Under the following conditions: - FOSC is selected as the PLL reference clock, - FOSC is configured as Bypass mode, and - The input reference clock frequency is in the range of 2 MHz to 4 MHz, the reset signal in the pre-divider logic of the PLL circuit is asynchronous to the pre-divided clock, which may introduce a metastable condition with low probability. Under specific pre-divider configurations (SCG_PLL0CFG.PREDIV = 0, corresponding to divide-by-1), this metastability can cause the divider logic to enter an abnormal state. In rare cases, this may result in either: - PLL failure to lock (SCG_PLL0CSR.PLLLK = 0), or - The valid flag failing to be asserted (SCG_PLL0CSR.VLD = 0).
CN	当满足以下条件时： - PLL 参考时钟选择 FOSC， - FOSC 配置为 Bypass 模式，且 - 输入参考时钟频率处于 2 MHz 至 4 MHz 范围内， 在 PLL 电路的预分频逻辑中，复位信号与预分频时钟为异步关系，较低概率下可能引发亚稳态。当预分频系数配置为 SCG_PLL0CFG.PREDIV = 0（不分频）时，该亚稳态可能导致分频逻辑进入异常状态。极少数情况下，此异常将导致： - PLL 无法锁定（SCG_PLL0CSR.PLLLK = 0），或 - 有效标志位无法置起（SCG_PLL0CSR.VLD = 0）。

Workaround

EN	Do not configure the PREDIV field in the PLL0CFG register to 0. Use PREDIV values ≥ 1 instead.
CN	禁止将 PLL0CFG 的 PREDIV 字段配置为 0。建议使用 PREDIV ≥ 1 的配置值。

2.17 ERR_SSI_3612116

Description

EN	Affected by an external RC oscillator, the current slew rate of an AK protocol SSI sensor may be relatively slow (refer to the sensor specification for details). During the current rise, the inm of the comparator may output 1 before the inh does so (such behavior does not conform to the AK protocol), resulting in inm glitches on both edges of the inh. To prevent the influence of inm glitches, the filter function of SSI must be enabled. However, due to the filter's limited bit width, unexpected glitches cannot be completely filtered out.
CN	受外部 RC 振荡器影响，AK 协议 SSI 传感器的电流转换速率可能较慢（具体参数请参考传感器规格书）。在电流上升过程中，比较器的 inm 信号可能先于 inh 信号输出高电平（此行为不符合 AK 协议规范），导致 inh 信号的上升沿和下降沿均出现 inm 毛刺。 为抑制 inm 毛刺的影响，必须使能 SSI 的滤波功能。但由于该滤波器的位宽限制，无法完全滤除所有异常毛刺。

Workaround

EN	To filter out all unexpected glitches, it is recommended to keep the frequency of the input functional clock of the SSI below 8 MHz.
CN	为确保彻底消除异常毛刺，建议将 SSI 模块的输入功能时钟频率控制在 8 MHz 以下。

2.18 ENET Errata

The following ENET-related errata for FC7300 are excerpted from Synopsys documents. For further information, visit the Synopsys website.

2.18.1 Synopsys ID: 4171346

Severity: B2-Medium

Title: Incorrect VLAN Tag Filter Fail Based Receive Queuing Ignores RA Bit of MAC_Packet_Filter Register

Impacted Configurations:

All DWC_ether_qos configurations in which Number of Receive Queues selected is greater than 1.

Defect Summary:

A received packet failing the VLAN Tag Filter is routed to the Receive Queue programmed in the VFFQ field with the VFFQE bit set to 1 in the MAC_RxQ_Ctrl4 register only if the RA bit of the MAC_Packet_Filter register is set to 1.

However, due to the defect, the received packet failing the VLAN Tag Filter is incorrectly routed to the Receive Queue programmed in the VFFQ field with the VFFQE bit set to 1, irrespective of the value of the RA bit.

There is no impact of this defect when the VTFE bit in the MAC_Packet_Filter register is set to 1, as the received packet that fails the VLAN Tag Filter is dropped in MAC.

Impacted Use Cases:

VLAN Filter Fail based Receive Queuing is enabled with both the RA and the VTFE bits of the MAC_Packet_Filter register set to 0.

Consequence:

The received packets that fail the VLAN Tag Filter are incorrectly routed to the Receive Queue programmed in the VFFQ field when the VFFQE bit is set to 1. Further, it may get routed to unintended Receive DMA (Virtual Machine in Virtualization applications) if static DMA mapping is enabled for the Receive Queue. This causes data loss.

Workaround:

The RA bit is used in Debug Mode to forward all the received packets to the software irrespective of the results of the MAC Filters. The Debug Mode feature of the VLAN Tag Filter fail based Receive Queuing must not be enabled in normal operation mode. The software must not set the VFFQE bit to 1 when the RA bit is set to 0.

Method of Reproducing:

- 1) Program both the VFFQ field and the VFFQE bit of the MAC_RxQ_Ctrl4 register to 1.
- 2) Program 16'h0001 in the VL field of the MAC_VLAN_Tag register.
- 3) Generate and send C-VLAN Tagged packet with VLAN Tag field 16'h0002 to the MAC receiver, so that received packet fails the VLAN Tag perfect filter.

Observe that the received packet is incorrectly forwarded to Receive Queue 1, instead of Receive Queue 0.

2.18.2 Synopsys ID: 3900281

Severity: B2-Medium

Title: MAC_System_Time_Higher_Word_Seconds Register Value Updated by DWC_ether_qos is Not Synchronized to the Transmit Timestamping Functions

Impacted Configurations:

EQOS configurations in which add IEEE 1588 higher word register and any of the following features are selected:

- Enable one-step timestamp feature
- Enable PTP timestamp offload feature
- Enable one step timestamp for PTP over UDP/IP feature

Defect Summary:

The MAC_System_Time_Higher_Word_Seconds register which indicates the 16-bit higher word seconds part of the system time is updated either by the application through the CSR slave interface, or internally by DWC_ether_qos when the MAC_System_Time_Seconds register rolls over. The updated value is provided to the one-step timestamping function.

However due to the defect, when the MAC_System_Time_Higher_Word_Seconds register is updated internally by DWC_ether_qos, the updated value is not synchronized to the one- step timestamping function in the MAC transmitter clock domain.

Impacted Use Cases:

When you program the MAC_System_Time_Seconds register to a value closer to the maximum value, it is likely to roll over. In real systems, this is not a valid use case as the roll over is expected to occur 136 years after January 01, 1970.

Consequence:

The one-step timestamping function inserts the incorrectly captured egress timestamp in the transmit PTP packets after the MAC_System_Time_Seconds register rolls over.

Workaround:

The application must read the MAC_System_Time_Higher_Word_Seconds register and write back the value it read, in the following cases:

- When DWC_ether_qos asserts the sbd_intr_o signal
 - To generate the timestamp seconds overflow interrupt. This is because you have programmed the TSIE field of the MAC_Interrupt_Enable register to 1.
 - Because TSSOVF field of the MAC_Timestamp_Status register and TSIS field of the MAC_Interrupt_Status register are 1.
- When you disable the interrupt by programming the TSIE field of the MAC_Interrupt_Enable register to 0 and the periodic polling of TSIS field of the MAC_Interrupt_Status register indicates that the value of the TSIS field is 1.

Method of Reproducing:

- 1) Initialize the MAC_System_Time_Seconds register to maximum value, that is 32'hFFFF_FFFF.
- 2) Program a value of 0 in the MAC_System_Time_Higher_Word_Seconds register.
- 3) Program the one-step timestamp function to operate in ordinary or boundary master mode by programming the fields of the MAC_Timestamp_control register as follows: SNAPTYPSEL = 2'b00 TSMSTRENA = 1'b1 TSEVNTENA = 1'b1.
- 4) Enable the Timestamp interrupt by programming the TSIE field of the MAC_Interrupt_Enable register to 1.

- 5) Wait for the generation of timestamp seconds overflow interrupt.
- 6) Generate and send a PTP sync packet by enabling one-step timestamping.

Observe that DWC_ether_qos performs one-step timestamping correctly but the system time-higher word seconds part in the original timestamp field of the PTP sync packet is 0, which is incorrect. The correct value is 1.

2.18.3 Synopsys ID: 3876182

Severity: B2-Medium

Title: MAC Unable to Identify PTP SYNC and Follow_Up Messages with Peer Delay Reserved Multicast Address in the 802.1AS Mixed Mode Operation

Impacted Configurations:

DWC_ether_qos configurations in which Enable IEEE 1588 Timestamp Support is selected.

Defect Summary:

The MAC operates in the IEEE 802.1AS mixed mode, when you program the AV8021ASMEN field of the MAC_Timestamp_Control register to 1. In the mixed mode, both Delay Request-Response and Peer Delay mechanisms are used together for PTP time correction. To capture the ingress timestamp, the MAC identifies the PTP SYNC and Follow_Up messages that contain PTP Peer Delay Reserved multicast destination address. You can program the mixed mode using any one of the following settings in the MAC_Timestamp_Control register fields:

- SNAPTYPSEL = 2'b01 and TSEVNTENA = 1'b0.
- SNAPTYPSEL = 2'b01, TSMSTRENA = 1'b0, and TSEVNTENA = 1'b1.

However due to the defect, when the MAC receives the PTP SYNC and Follow_Up messages with PTP Peer Delay Reserved multicast destination address, it does not identify the packets as PTP packets. The MAC identifies the received PTP SYNC and Follow_Up messages only if it contains PTP primary multicast address.

Impacted Use Cases:

The MAC operates in the IEEE 802.1AS mixed mode and receives PTP SYNC and Follow_Up messages.

Note: In the IEEE 802.1AS mixed mode, both Delay Request-Response and Peer Delay mechanisms are used together for PTP time correction.

Consequence:

The MAC does not capture the ingress timestamp for PTP SYNC and Follow_Up messages that are received with PTP Peer Delay Reserved multicast destination address. This results in inaccuracies in the maintained system time. The slave node is unable to compute and perform the time correction.

Workaround:

The IEEE 802.1AS mixed mode is not a general use case. The time correction can be performed by using either Delay Request-Response or Peer Delay mechanism. However, if mixed mode is required the application must program the TSENALL field of the MAC_Timestamp_Control register to 1, to enable the MAC to capture the timestamp for all the received packets. The software must identify the PTP SYNC and Follow_Up messages and associate the timestamp status provided by the MAC.

Method of Reproducing:

- 1) Enable the IEEE 802.1AS mixed mode by programming the MAC_Timestamp_Control register as follows:

- AV8021ASMEN = 1
- SNAPTYPSEL = 2'b01
- TSEVNTENA = 1'b0

- 2) Generate and send PTP SYNC message with PTP Peer Delay Reserved multicast destination address to the MAC receiver.

Observe that DWC_ether_qos incorrectly updates the PMT and TSA fields to 0 in the receive packet status. This indicates that no PTP message is received and no timestamp is available. Therefore, the MAC does not provide the receive timestamp status to the application.

2.18.4 Synopsys ID: 3873623

Severity: B3-High

Title: Un-Correctable FSM Timeout Safety Interrupt Incorrectly Generated for Long Waiting FSM States in the Normal Operation

Impacted Configurations:

DWC_ether_qos configurations in which Enable All Automotive Safety Features is selected.

Defect Summary:

When an FSM must wait in a state for a longer duration based on the dependent events, the state of the FSM is categorized as a safe state and is excluded from monitoring for timeout. The FSM can wait in the following states for a longer duration. These are categorized as safe states and do not trigger un-correctable FSM timeout safety interrupt:

- When you enable the transmit DMA by programming the ST bit of the DMA_CH(#i)_Tx_Control register to 1 but do not enable the MAC transmitter, that is TE bit of the MAC_Configuration register is 0, the transmit packet fetched by the DMA is provided to the MAC engine and the Transmit Bus Unit (TBU) and Transmit Frame Controller (TFC) FSMs become active. The TBU and TFC FSMs which are in the MAC transmitter clock domain, transition to TBU_WAIT_FOR_EOF and TFC_WAIT_FOR_GNT states respectively, and wait for the MAC transmitter to be enabled, for transmitting the packet. This wait can be longer than the programmed FSM timeout limit.
- When the MAC enters the transmit LPI mode, the Transmit Protocol Engine (TPE) FSM transitions to the TPE_LPI state and starts driving the LPI pattern on the line. The LPI pattern is driven until the transmit LPI is disabled in the manual or automatic mode (LPIEN bit of the MAC_LPI_Control_Status register becomes 0) or by resuming the traffic in automatic mode. The wait in the TPE_LPI state can be longer than the programmed FSM timeout limit.

However, due to the defect, the time to

- Enable the MAC transmitter in scenario 1
- Exit the transmit LPI in scenario 2

exceeds the programmed FSM timeout limit. Therefore, the MAC generates the spurious un-correctable FSM timeout safety interrupt.

Impacted Use Cases:

- The application enables the transmit DMA before the MAC transmitter and the time duration between them is greater than the programmed FSM timeout limit.
- The MAC remains in the transmit LPI state for a time duration greater than the programmed FSM timeout limit.

Consequence:

The spurious un-correctable FSM timeout safety interrupt triggers an unintended recovery process which is time-consuming. That is, issuing of soft or hard reset followed by reconfiguration, which impacts performance.

Workaround:

After enabling the MAC transmitter, the application must enable the transmit DMA. The application must disable the FSM timeout function immediately after receiving the transmit LPI entry interrupt. Also, the application must enable the FSM timeout function after the transmit LPI exit interrupt.

Note: The FSM timeout for all FSMs is disabled for the duration of transmit LPI. This workaround can be used only when the application can disable FSM timeout before the programmed FSM timeout limit is reached.

Method of Reproducing:

- 1) Enable the MAC transmitter by programming the TE bit of the MAC_Configuration register to 1.
- 2) Enable automated Transmit LPI entry or exit by programming the LPIEN and LPITXA bits of MAC_LPI_Control_Status register to 1.
- 3) Enable FSM timeout limit of 1 second by programming.
 - TMOUTEN field of the MAC_FSM_Control register to 1.
 - NTMRMD field of the MAC_FSM_ACT_Timer register to 4'h6.
- 4) Do not send any traffic for more than 1-second duration.

Observe that DWC_ether_qos incorrectly generates un-correctable FSM timeout safety interrupt on the sbd_sfty_ue_intr_o output port. Also, DWC_ether_qos writes 1 in the TTES bit of the MAC_DPP_FSM_Interrupt_Status register indicating to the application that an FSM timeout has occurred in the MAC transmitter clock domain.

2.18.5 Synopsys ID: 3683170

Severity: B2-Medium

Title: Incorrect Routing of Rx Packet or Incorrect Splitting of Header Payload When CDC Delay is Different in the Asynchronous FIFOs Between Receiver Clock and Application Clock

Impacted Configurations:

EQOS-DMA, EQOS-AHB, EQOS-AXI, and EQOS-AXI4 configurations in which Use Single- Port RAM and Enable Frame Pre-Emption Support features are selected in the configuration.

Defect Summary:

The MAC receiver provides the header payload splitting, and the Rx DMA routing information to the MTL Rx queue controller through the header status. For every packet, the MAC receiver provides a header status.

When a received packet is pre-empted, the MAC receiver pushes a dummy status in the header status asynchronous FIFO between receiver clock and application clock. for every fragment, except the one for which MAC receiver provides the Header status. In case of Rx queue overflow, and the fragment is dropped, the corresponding header status is flushed from header status Asynchronous FIFO at the end of fragment reception.

In case of Rx queue overflow for a packet or delays in the CDC synchronizers in the MAC receiver to Rx DMA path, DWC_ether_qos correctly performs the header-payload splitting or routing the received packet to the Rx DMA. However

due to the defect, when all the following conditions are met, the routing of the received packet and header-payload splitting, is incorrect:

- Rx queue overflows for any earlier fragment of the ongoing pre-emption packet and the current continuation fragment must be dropped without writing the word in the Rx queue.
- The dummy header status or the valid header status from the MAC receiver is written to header status Asynchronous FIFO around the same time when the last word of the fragment (EOF) is written to packet data Asynchronous FIFO and the latter has lesser CDC synchronization delay.

When the MTL Rx queue controller observes EOF word in the packet data asynchronous FIFO, it places read request for EOF and header status simultaneously. The EOF is read out correctly but read request for header status is not accepted because the Header Status Asynchronous FIFO is empty due to larger CDC synchronization delay. The Header status remains in the header status Asynchronous FIFO and is associated with subsequent pre-emption fragment or express packet. Because of the cascading effect Header status of all subsequent pre-emption fragments or express packets is of the previous pre-emption fragment or express packet.

Impacted Use Cases:

The pre-empted packets are received by the MAC receiver and the application bandwidth is not enough resulting in overflow of the pre-emption Rx queue.

Consequence:

After the defect occurs, the header status associated with all subsequent pre-emption fragments or express packets is from previous pre-emption fragment or express packet. This results in incorrect header-payload splitting or Rx DMA routing for subsequent packets. The application connected to the Rx DMA drops or performs incorrect assembly of the packets. Data loss or retransmission impacts performance.

Workaround:

The defect is not observed when the bandwidth on the application side is enough to prevent the Rx queue overflow. The system design must ensure this. The software must program larger size for the pre-emption Rx queues if heavy traffic is observed.

There is no workaround if these requirements cannot be met by the system design or software.

Method of Reproducing:

One of the requirements for the defect to occur is different CDC synchronization delays for packet data and header status asynchronous FIFOs. Therefore, the defect cannot be easily reproduced in simulations.

2.18.6 Synopsys ID: 3413311

Severity: B2-Medium

Title: Incorrect Handling of Application Bus Error in Certain Boundary Conditions

Impacted Configurations: All DWC_ether_qos DMA configurations.

Defect Summary:

When an application bus error occurs, the corresponding Transmit or Receive DMA Channel performs the following tasks, before transitioning to the stop state:

- 1) Terminates the ongoing transmit packet fetching

- 2) Accept the pending transmit status
- 3) Flush the remaining part of the received packet
- 4) Generate Fatal Bus Error interrupt.

These steps ensure that the application can restart/reconfigure the Transmit or Receive DMA channel without the need for a software reset.

However due to the following defects, the DMA Channel does not handle the Bus Error gracefully and requires Software Reset for recovery:

- 1) When you program the OSF field of the DMA_CH(#i)_TX_Control register to 1, that is, the Operating on Second Packet (OSP) mode is enabled, the transmit status for the previous packet is pending to be written to the descriptor memory; Bus Error occurs on the non-last descriptor of the current packet. The Transmit DMA does not read and discard the pending transmit status of these two packets.
- 2) When the Bus Error occurs on the first data word of the transmit packet of the pre-emption queue, the MAC Transmitter does not transmit single byte pre-emption fragment of the packet that encountered the Bus Error and the subsequent pre-emption and express packets.
- 3) When Bus Error occurs on the Receive Descriptor fetch to the Descriptor Cache, the Receive DMA indicates a Bus Error to the Descriptor Cache. As a result, descriptor burst transfer completion is indicated to the Receive DMA. As the Bus Error and descriptor burst transfer complete signal do not coincide, which is incorrect, the Receive DMA misses the Bus Error.
- 4) When the Bus Error occurs on the first data word of the transmit packet, the start of packet is not indicated in the single dummy word, as expected. So, the start and end of packet indication is pushed to the Transmit Queue.

Impacted Use Cases:

The AXI Bus/Slave/Interconnect, to which the DWC_ether_qos is interfaced, supports bus error generation for incorrect accesses or transfers.

Consequences:

- 1) The STXSTSF and TXSTSFSTS fields of the MTL_TxQ(#i)_Debug register are not cleared. If the Fatal Bus Error Interrupt Service Routine (ISR) waits for all Debug Status registers in DWC_ether_qos to be cleared, before restarting the impacted DMA, it cannot come out of ISR. The recovery is only through software reset.
- 2) The MAC Transmitter does not transmit subsequent packets from any Transmit Queue. The recovery is only through Software Reset.
- 3) As the Bus Error is neither detected, nor reported by DWC_ether_qos, the software is unable to take the required corrective action, which can lead to serious system errors.
- 4) The MTL Transmit Read Controller does not transmit subsequent packets from any Transmit Queue. The recovery is only through software reset.

Workaround:

The software must use the Bus Error recovery mechanism of issuing Software Reset and reconfiguring DWC_ether_qos. This impacts only the performance.

Method of Reproducing:

1) Defect 1:

- A. Program OSF field of the DMA_CH0_TX_Control register to operate the Transmit DMA Channel 0 in Operate on Second Packet mode.
- B. Program the Transmit Queue 0 size to accommodate 2 packets of size 1518 bytes each.
- C. Generate and send 2 packets of size 1518 bytes each to the MAC Transmitter. Distribute the second packet data in the buffers of 2 Transmit descriptors.
- D. Generate Bus Error for the first Descriptor write-back of the second packet; the transmit status of the first packet must be pending.

Observe that Fatal Bus Error interrupt is correctly generated. However, STXSTSF and TXSTSFSTS fields of the MTL_TxQ(#i)_Debug register are not cleared, which is an incorrect behavior.

2) Defect 2:

- A. Generate and send 2 packets of size 1518 bytes. First packet to Pre-emption Transmit Queue 0 and second packet to Express Transmit Queue 1.
- B. Generate Bus Error for first data word of the first packet.

Observe that Fatal Bus Error interrupt is correctly generated. However, the MAC Transmitter does not transmit both the packets, which is an incorrect behavior.

3) Defect 3:

- A. Generate and send 4 packets of size 1518 bytes each to Receive DMA Channel 0.
- B. Generate Bus Error for the descriptor read burst.

Observe that the Fatal Bus Error interrupt is not generated. All 4 packets are transmitted without errors, which is an incorrect behavior.

4) Defect 4:

- A. Generate and send 2 packets of size 1518 bytes each; first packet to Transmit Queue 0 and second packet to Transmit Queue 1.
- B. Generate Bus Error for first data word of the first packet.

Observe that the Fatal Bus Error interrupt is correctly generated, but the MAC Transmitter does not transmit the corrupted first packet and the error-free second packet, which is an incorrect behavior.

2.18.7 Synopsys ID: 3380358

Severity: B2-Medium

Title: Pre-Emptable Traffic is Incorrectly Blocked Even When the Available Express Traffic Cannot be Scheduled

Impacted Configurations:

DWC_ether_qos configurations in which Enable Enhancements to Scheduling Traffic (EST) and Frame Pre-Emption Support are selected.

Defect Summary:

When both IEEE 802.1Qbv (EST) and IEEE 802.3br (frame pre-emption, FPE) are enabled, IEEE 802.1Qbu describes use-cases on how the express and pre-emption traffic must be scheduled with minimum or no interference with each other. The use case

- R.3 mentions about minimum interference with express traffic limited to maximum frame pre-emption fragment, which can be up to 128 bytes.
- R.4 mentions about no interference to express traffic by providing dedicated hold window where pre-emption traffic cannot be scheduled.

However due to the defect, when the express Transmit Queue has express packet available, the pre-emption Transmit Queues are blocked. This block is irrespective of whether the express packet can be scheduled, or not. For example,

- When GCL gates of Express Transmit Queues are closed
- When Credit Based Shaper (CBS) Scheduler cannot schedule the express packet due to negative credit.

When a packet from the pre-emption Transmit Queue is ongoing, it is pre-empted, and the next fragment is not scheduled until all available express packets are transmitted. This shows that DWC_ether_qos does not adhere to the use cases R.3 and R.4 of IEEE 802.1Qbu.

Impacted Use Cases:

When Express Traffic controlled by EST and Frame Pre-emption Traffic are enabled together.

Consequence:

The pre-emption Transmit Queues are blocked when Express Packets are available, even if the Express packets cannot be scheduled. This results in inefficient utilization of the available bandwidth.

Workaround:

Not required. Impact is only on the performance of pre-emption traffic.

Method of Reproducing:

- 1) Program a GCL with 3 slots such that, in the first slot, the gates of all Express Transmit Queues are closed.
- 2) Generate and send one packet each, to express and pre-emption Transmit Queues.

Observe that the packet from pre-emption Transmit Queue is not transmitted in the first slot even though express packet cannot be transmitted; GCL gates of all Express Transmit Queues are closed.

2.18.8 [Synopsys ID: 3379533](#)

Severity: B2-Medium

Title: Larger Interpacket Gap When WRR Algorithm is Selected for Scheduling Packets from Transmit Queues

Impacted Configurations:

Configurations other than EQOS-CORE, in which Enable Enhancements to Scheduling Traffic (EST) feature is selected.

Defect Summary:

The Weighted Round Robin (WRR) algorithm schedules packets from highest numbered transmit queue up to Transmit Queue 0. In a service round, the number of packets scheduled from a transmit queue depends on the weights programmed in the ISCQW field of the MTL_TxQ(#i)_Quantum_Weight register. Transmit queues whose number is greater than the transmit queue being serviced, are blocked from participating in the scheduling, if they are already

served or do not have packets during the current service round. When the new service round begins, the scheduler restarts the service from the highest numbered transmit queue.

However due to the defect, when the number of packets in the Transmit Queue 0 is less than its programmed weight, the scheduler schedules the available packets and incorrectly waits for a fixed duration, before ending the current service round. This wait is independent of the availability of packets in the higher numbered transmit queues. This results in larger interpacket gap (IPG) than expected, even when packets are available in the higher numbered transmit queues.

Impacted Use Cases:

The WRR algorithm is enabled for scheduling the packets from transmit queues and the programmed weight of the Transmit Queue 0 is greater than 1.

Consequence:

Larger interpacket gap (IPG) than expected, which impacts performance or throughput.

Workaround: Not Applicable.

Method of Reproducing:

- 1) Program the SCHALG field of the MTL_Operation_Mode register to 0. This selects WRR algorithm for scheduling packets from the transmit queues.
- 2) Program a weight of 2 for Transmit Queue 0 and a weight of 1 for all the other transmit queues. To program the weights, use the ISCQW field of MTL_TxQ(#i)_Quantum_Weight register.
- 3) Generate and send 2 packets each to Transmit Queue 1 and higher numbered queues; send 1 packet to Transmit Queue 0.

Observe that the MAC transmits the first packet from every transmit queue with an IPG of 12 bytes, as expected. However, the IPG between the first packet from Transmit Queue 0 and second packet from the highest numbered transmit queue is greater than 12 bytes, which is incorrect.

2.18.9 Synopsys ID: 3368939

Severity: B2-Medium

Title: Incorrect Packet Size Related Head-Of-Line Blocking Error When Gates of Gate Control List are Closed

Impacted Configurations:

DWC_ether_qos configurations in which Enable Enhancements to Scheduling Traffic (EST) feature is selected.

Defect Summary:

The packet size related Head-Of-Line Blocking error is detected when the gate of the Gate Control List (GCL) for the Transmit Queue is open and the packet cannot be scheduled as the packet size is greater than slot time. The packet size related Head-Of-Line Blocking (HLBF) error is reported when packet cannot be scheduled for two complete iterations of GCL.

However due to the defect, incorrect HLBF error is detected when a packet from a Transmit Queue is available for scheduling but the GCL gates for that Transmit Queue are closed for two complete iterations of GCL. This is because the gate open event was not considered when HLBF monitoring started during the availability of packet in the Transmit Queue.

The DWC_ether_qos now supports additional programming control to reduce the HLBF monitoring interval to one complete iteration of GCL (QHLBF field in the MTL_EST_Control register).

Impacted Use Cases:

EST scheduler is enabled and gates of a Transmit Queue are closed for two complete iterations of GCL.

Consequence:

Due to the incorrect detection of HLBF error, the packet is incorrectly dropped if the DDBF field of the MTL_EST_Control register is set to 0. This results in data loss.

Workaround:

The software must program the DDBF field of the MTL_EST_Control register to 1. This avoids data loss. However, the packet is not dropped when a genuine HLBF error occurs; blocking the subsequent packets in the Transmit Queue, which requires the software to provide new GCL.

Method of Reproducing:

- 1) Program a GCL with 4 slots (each slot-time must be able to send a packet of 64 bytes). Also, the gates for express Transmit Queue 1 must be closed for all slots.
- 2) Generate and send a packet of size 128 bytes to the express Transmit Queue 1.

Observe that, after the completion of 2 iterations of GCL, the HLBF status field of the MTL_EST_Status register is incorrectly set.

2.18.10 Synopsys ID: 3340890

Severity: B2-Medium

Title: Flexible PPS Output Incorrectly Generated on Target Time Error or When Fine Correction Method is Enabled in the External Timestamping Mode, and Sub-Nano Second not Supported in the Binary Rollover Mode

Impacted Configurations:

DWC_ether_qos configurations in which Flexible Pulse-Per-Second Output, Sub-Nano Second Support, or Both options for IEEE 1588 System Time Source is selected.

Defect Summary:

DWC_ether_qos reports the Target Time error when the programmed target time is already elapsed; that is, programmed target time is lesser than system time. The PPS output is not generated in this error condition.

When Sub-Nano Second support is selected in the configuration, it is used to improve the accuracy of timestamping even in Binary Rollover mode (TSCTRLSSR field of the MAC_Timestamp_Control register is set to 0).

When Both options for IEEE 1588 System Time Source is selected in the configuration, the fine correction method (TSCFUPDT field of the MAC_Timestamp_Control register is set to 1) is not effective in the external timestamping mode (ESTI field of the MAC_Timestamp_Control register is set to 1).

However due to the defect,

- The PPS output is incorrectly generated even when Target Time Error occurs.
- The Sub-Nano Second accuracy cannot be used in binary rollover mode.

- The PPS output (width and interval) is incorrect when the fine correction method (TSCFUPDT field of the MAC_Timestamp_Control register is set to 1) is enabled in the external timestamping mode (ESTI field of the MAC_Timestamp_Control register is set to 1).

Impacted Use Cases:

The following use cases are impacted:

- Flexible PPS Output feature is enabled and CSR access delays are such that Target Time Error can occur when Target Time register is programmed
- The Sub-Nano Second accuracy is used in Binary Rollover mode
- The software programs the internal Timestamping control fields while operating in External Timestamping mode, or the software does not clear the Internal Timestamping control fields before switching to the External Timestamping mode

Consequence:

The defect can have the following consequences:

- The incorrect Flexible PPS output generated during Target Time Error can trigger an unintended implementation specific event.
- The timestamp captured or inserted in the PTP packets in the Binary Rollover Mode can be less accurate.
- The incorrect Flexible PPS Output generated due to fine correction being used in the External Timestamping mode can make the dependent implementation specific logic to go out of sync.

Workaround:

Following are the workarounds:

- The software must accurately compute the CSR access delays. This avoids Target Time Error when programming the Target Time register.
- Not applicable as the inaccuracies due to Sub-Nano Second not used in Binary Rollover mode is insignificant.
- The software must not program Internal Timestamping control fields while operating in External Timestamping mode.

Alternatively, the software must clear the Internal Timestamping control fields before switching to the External Timestamping mode.

Method of Reproducing:

- 1) Connect a 50MHz clock to the clk_ptp_ref_i input.
- 2) Enable the Flexible PPS Output feature by programming the PPSEN0 field to 1 and PPSCTRL_PPSCMD field to 2, of the MAC_PPS_Control register.
- 3) Program the MAC_PPS0_Target_Time_Seconds and MAC_PPS0_Target_Time_Nanoseconds registers with values lesser than the current system time. The current system time is present in the MAC_System_Time_Seconds and MAC_System_Time_Nanoseconds registers.
- 4) Program the MAC_Timestamp_Addend register to a non-zero value.
- 5) Program the ESTI and TSCFUPDT fields of the MAC_Timestamp_Control register to 1.
- 6) Program the TSADDREG field of the MAC_Timestamp_Control register to 1, to start the fine correction.

- 7) Program the MAC_PPS0_Target_Time_Seconds and MAC_PPS0_Target_Time_Nanoseconds registers with values greater than the current system time. The current system time is present in the MAC_System_Time_Seconds and MAC_System_Time_Nanoseconds registers.
- 8) Program the PPSINT0 field of the MAC_PPS0_Interval register to 4. This indicates PPS interval of 5 cycles of the clk_ptp_ref_i signal.
- 9) Program the PPSWIDTH0 field of the MAC_PPS0_Width register to 3. This indicates PPS pulse duration of 4 cycles of the clk_ptp_ref_i signal.

Observe that, after step 3, incorrect Flexible PPS output is generated.

After step 9, observe pulse train of incorrect width (not equal to 80ns) and incorrect interval (not equal to 100ns) on the Flexible PPS output.

2.18.11 Synopsys ID: 3338201

Severity: B2-Medium

Title: Unintended CSR Byte Update due to Missing Byte-Enable Qualification

Impacted Configurations:

DWC_ether_qos configurations in which more than 1 Receive Queues are selected.

Defect Summary:

When the software writes a byte or a half-word to a CSR register, only the corresponding bytes of the register are updated.

However due to the defect, when the software writes 1, 2, or 3 bytes or higher half word to the MAC_Indir_Access_Ctrl register, the OB field (Operation Busy) of the MAC_Indir_Access_Ctrl register is updated to 1.

Impacted Use Cases:

The software writes a byte or a half-word to update the MAC_Indir_Access_Ctrl register.

Consequence:

When the value of the OB field (Bit[0]) of the MAC_Indir_Access_Ctrl is 1, it triggers unintended write or read access to the indirect register. If the value of COM field (Bit[1]) of the MAC_Indir_Access_Ctrl is 0, the invalid Ethernet Type value is written to the MAC_TMRQ_Regs(#i) indirect register. So, the received packets are routed to incorrect Receive Queue/DMA Channel, resulting in loss of data.

Workaround:

The software must perform only word (32-bit) wide writes to the MAC_Indir_Access_Ctrl register.

Method of Reproducing:

- 1) Program 32'h0001_0700 in the MAC_Indir_Access_Data register.
- 2) Transfer the data in the MAC_Indir_Access_Data register to the indirect register, MAC_TMRQ_Regs0 by programming 32'h0000_0001 in the MAC_Indir_Access_Ctrl register.
- 3) Program 32'h0001_0701 in the MAC_Indir_Access_Data register.
- 4) Perform write access to only byte 1 (byte enables 4'h2) of the MAC_Indir_Access_Ctrl register by programming 32'h0000_0001.

- 5) Generate and send packet with Ethernet Type field, 16'h0700 to the MAC receiver.

Observe that the received packet is incorrectly routed to Receive Queue 0, instead of Receive Queue 1.

2.18.12 Synopsys ID: 3309847

Severity: B2-Medium

Title: MAC Incorrectly Discards the Received Packets When Preamble Byte Does not Precede SFD or SMD

Impacted Configurations:

DWC_ether_qos configurations in which Enable Frame Preemption Support is selected.

Defect Summary:

The IEEE 802.3 standard states that, in MII/GMII modes, the byte preceding the SFD (0xD5), SMD-S (0xE6, 0x4C, 0x7F, or 0xB3), or SMD-C (0x61, 0x52, 0x9E, or 0x2A) byte can be a non-PREAMBLE byte or there can be no preceding preamble byte. The MAC receiver must successfully receive a packet without any preamble (0x55) byte preceding the SFD, SMD-S, or SMD-C byte.

However due to the defect, in configurations where frame preemption is enabled, when preamble byte does not precede the SFD, SMD-S, or SMD-C byte, the received packet is discarded by the MAC receiver. This is because, the start-of-packet detection logic of the MAC receiver incorrectly checks for a preamble byte.

Impacted Use Cases:

The frame preemption feature is enabled in the configuration and a packet is received without a preamble byte preceding the SFD, SMD-S, or SMD-C byte. The preamble byte might be absent due to any of the following reasons:

- Remote transmitter transmits packets without preamble (chip-to-chip applications).
- Preamble byte corrupted in transit.
- Repeater removes the preamble byte.

Consequence:

The MAC receiver discards the received packet for which preamble byte does not precede the SFD, SMD-S, or SMD-C bytes, resulting in loss of data. However, applications that support retransmission can retransmit the packet. Retransmissions impact performance.

Workaround:

If the remote transmitter (chip-to-chip application) does not transmit preamble byte, configure it to transmit at the least one preamble byte preceding the SFD, SMD-S, or SMD-C byte.

If the preamble byte is corrupted in transit, it is a transient issue; only a few packets are impacted. No workaround is required.

Workaround not available for other cases.

Method of Reproducing:

Generate and send a packet to the MAC receiver without preamble bytes or corrupt the preamble byte that precedes the SFD byte.

Observe that the MAC receiver discards the received packet.

2.18.13 Synopsys ID: 3294970

Severity: B3-High**Title:** DMA Halts Operation or Incorrect or no Corrupt Packet on Detecting Uncorrectable Safety Error**Impacted Configurations:**

DWC_ether_qos configurations in which Enable All Automotive Safety Features or SECDED ECC Protection for external memories is selected.

Defect Summary:

When an uncorrectable safety error is detected in DWC_ether_qos or at its application interfaces, an uncorrectable safety interrupt is generated and:

- The impacted transmit packet is either dropped or terminated with CRC error, so that it is dropped by the remote receiver
- The impacted receive packet is either dropped or forwarded to the application with an error indication in the Receive status

However due to the defect,

- When OSF (operate on second frame) mode is enabled in the DMA_CH(#i)_Tx_Control register and safety error occurs on data buffer read for the second packet (corresponding to first or intermediate descriptors, but not the last descriptor), Transmit DMA neither processes the subsequent packets nor updates the status for the transmitted packets. Software reset is the only recovery mechanism.
- When safety error occurs on the CD bit (context descriptor) of the Transmit descriptor and control words or packet data is yet to be provided to MTL, the Transmit DMA halts in "waiting for status" state. Transmit DMA neither processes the subsequent packets nor updates the status for the transmitted packets. Software reset is the only recovery mechanism.
- When safety error occurs for the first or intermediate TSO segment data read access, Transmit DMA does not provide safety error status in the descriptors corresponding to the intermediate TSO segments.
- When safety error occurs on data buffer read corresponding to the last descriptor of the packet, the packet is transmitted without corruption because the ati_err_i signal is not provided by DMA to MTL.
- When safety error occurs on data buffer read (at very end of the transfer) and in the meanwhile, if data buffer read from the next DMA starts immediately, the packet transmitted from the other Transmit DMA is corrupted. This is a corner-case scenario because of interleaving of read operations or back-to-back transfers on the AXI, for different Transmit DMAs.
- When ECC error is injected towards end of packet word on Transmit Queue read operations, the packet is transmitted on the line without corruption as mti_err_i signal is not provided by MTL to MAC. This is because even though ECC error is detected on the Transmit Queue read, it is informed to the Transmit Queue write side to assert ati_err_i signal; in this case the packet transmission is already over on the Transmit Queue write side, and ati_err_i is not inserted.
- Transmit DMA behavior is incorrect when safety error occurs soon after control words (partial or complete) are sent to MTL. The current packet is not initiated on the line as packet data is not sent to MTL, but safety error is correctly updated in the Transmit status. The read side of the Transmit Queue interprets the already received control words as belonging to the next packet, impacting its processing.
- The DPP debug error injection mechanism requires the packet to be transmitted or received. To check the checker operation, the DPP debug error injection mechanism injects error in the checker path and not in the

packet data. As the debug error is not injected in the packet, there is no scope for the Transmit packet to corrupt and the Transmit or Receive packet status to be updated with safety error status. However, the Transmit packet transmitted on the line is corrupted and flags a safety error in the Transmit status. A similar behavior is seen for received packet also.

Impacted Use Cases:

When safety features are enabled in the configuration.

Consequences:

Following are the consequences:

- The Transmit DMA halts its operation. Recovery is through software reset. The reset and subsequent reconfiguration impacts performance.
- When packets with uncorrectable safety error are not corrupt, the remote end receiver processes and forwards these packets to the software. The software performs unintended operations on the incorrect packet data.
- Corruption of packet results in loss of data. This impacts performance in applications that support detection of data loss and retransmission.
- When corrupt debug packets are received by the software, it might interpret this as uncorrectable error, raising false alarm.

Workaround: None.

Method of Reproducing:

- 1) Generate and send a packet to the MAC transmitter.
- 2) Enable CRC/Pad insertion in MAC for this packet.
- 3) Inject 2-bit ECC error on the EOF word when the packet is read from MTL Tx Queue.

Observe that the transmit packet does not have CRC error, which is an incorrect behavior

2.18.14 Synopsys ID: 3290261

Severity: B2-Medium

Title: Scheduled Transmit Packet not Sent in the Allotted Slot or the Remaining Fragment of a Preempted Packet Incorrectly Dropped due to Scheduling Timeout in the EST GCL

Impacted Configurations:

DWC_ether_qos configurations in which Enable Enhancements to Scheduling Traffic (EST) and Use Single-Port RAM options are selected.

Defect Summary:

The EST (Enhancements to Scheduled Traffic) scheduler schedules the packet for transmission in a GCL slot, only when the sum of packet size (adjusted for change in the packet size due to offloads enabled in the MAC), fixed packet overhead, and the scheduler delay is less than the duration available in the GCL slot.

Also, when the first fragment or a few fragments of the preempted packet are transmitted, the MAC does not drop the remaining fragment because of the scheduling timeout or expiry due to excessive express packet transmission.

However due to the defect,

- The boundary size packets (packets that almost or exactly fit the GCL slot) are not scheduled in the expected GCL slot. This is because, the GCL slot is generated in the PTP clock domain and the scheduler operates in the application clock domain. The sync delay of this asynchronous path introduces minor inaccuracies in the computation of the time available in the GCL slot. This results in the packet not being scheduled in the allotted GCL slot.
- The remaining fragment of the preempted packet is dropped because of scheduling timeout or expiry due to excessive express packet transmission.

Impacted Use Cases:

- The GCL slot intervals are computed and programmed to exactly fit the packets to be scheduled by the EST scheduler.
- The DFBS field of the MTL_EST_Control register is set to 1, to drop packets that encounter scheduling error.

Consequences:

- The packet is not scheduled in the expected GCL slot. Even if the packet is subsequently scheduled and transmitted, the remote receiver drops the packet, as the packet is not received at the expected time. This results in loss of data.
- The remaining fragment of the preempted packet is dropped. The remote receiver drops the packet as it does not receive the complete packet. This results in loss of data.

Workaround:

Software can implement the following workarounds:

- Program the GCL slot interval to be larger than the sum of the packet size (adjusted for packet size change due to offloads enabled in MAC), fixed packet overhead, and scheduler delay, by at least the maximum sync delay between the PTP clock domain and application clock domain (for example, 1 clock period of PTP clock and 3 clock periods of application clock).
- Disable the dropping of packets that encounter scheduling error, by programming the DFBS field of the MTL_EST_Control register to 0.

Method of Reproducing:

- 1) Program a GCL list with 25 slots such that the packet fits exactly in each slot (without considering the sync delay in the computation)
- 2) Generate and send 25 packets to the MAC transmitter

Observe that, one of the transmit packets is not scheduled in the expected GCL slot. The number of slots after which this issue is observed depends on the phase and frequency relationship between the PTP clock and the application clock.

2.18.15 Synopsys ID: 3290231

Severity: B2-Medium

Title: Transmit MMC Counters Incorrectly Updated for Frame Preemption Packets When the Accumulated Byte Counter Overruns

Impacted Configurations:

DWC_ether_qos configurations in which Enable Frame Preemption Support and Transmit MMC counters that are updated based on accumulated byte count status, are selected.

Defect Summary:

The MAC accumulates the status of all fragments of a preempted packet before updating the Transmit MMC counters.

However due to the defect, when the sum of the bytes of all fragments of a preempted packet exceeds the Jabber Timeout limit of 16383 bytes, the Transmit MMC counters that are updated based on accumulated byte count status are incorrectly updated. Because of an overflow in the accumulated byte counter, the accumulated byte counter incorrectly indicates only the bytes that exceed the 16383 limit, instead of saturating at 16383 bytes.

Impacted Use Cases:

The statistics captured in the Transmit MMC counters is used by the software.

Consequence:

The Transmit MMC counters are incorrectly updated, resulting in incorrect statistics. This impacts any dependent function implemented by the software.

Workaround:

The software can implement statistics counters and update them based on the Transmit status provided by the MAC.

Method of Reproducing:

- 1) Generate and send transmit packet of size 16384B through the Transmit Queue 0 which is a preemption queue.
- 2) While the packet from Transmit Queue 0 is being transmitted, generate and send transmit express packet through Transmit Queue 1 such that it preempts the packet transmitted from Transmit Queue 0.
- 3) Read the Tx_Octet_Count_Good_Bad register after both the fragments of the preempted packet are transmitted from Transmit Queue 0.

Observe that the Tx_Octet_Count_Good_Bad counter incorrectly indicates a count of 1 byte, instead of 16383 bytes.

2.18.16 Synopsys ID: 3288309

Severity: B2-Medium

Title: Transmission Status not Updated in the Status Register for Back-to-Back Frame Preemption Verify or Respond Packets

Impacted Configurations:

DWC_ether_qos configurations where Enable Frame Preemption Support feature is selected.

Defect Summary:

The software can trigger Frame Preemption Verify or Respond packet by setting the SVER or SRSP fields respectively, of the MAC_FPE_CTRL_STS register. The MAC transmits the Frame Preemption Verify or Respond packet and updates the TVER or TRSP fields respectively, of the MAC_FPE_CTRL_STS register.

However due to the defect, when Frame Preemption Verify and Respond packets are triggered by the software simultaneously or within a very short duration, the transmission status of the already triggered packet is not updated in the MAC_FPE_CTRL_STS register. When both are triggered simultaneously, the Frame Preemption Respond packet is transmitted first. So, the transmission status of Frame Preemption Respond packet is not updated in the status register.

Impacted Use Cases:

Software triggers the Frame Preemption Verify and Respond packets simultaneously or within a very short duration.

Consequence:

As per the defined protocol, the software must wait for the completion of the already triggered Frame Preemption Verify or Respond packet, before triggering the same packet again. As the transmission status is not updated in the status register, the software cannot trigger the packet again, for which transmission status is not yet updated. The MAC is unable to exchange or synchronize the Frame Preemption support status with the remote receiver.

Workaround:

The software must not trigger back-to-back Frame Preemption Verify and Respond packets. Software must wait for the completion of the already triggered packet before triggering the next packet.

Method of Reproducing:

- 1) Program the SVER field of the MAC_FPE_CTRL_STS register to 1.
- 2) Program the SRSP field of the MAC_FPE_CTRL_STS register to 1.

Observe that the TVER field of the MAC_FPE_CTRL_STS (status) register is not set to 1, even though the Frame Preemption Verify packet is transmitted correctly.

2.18.17 Synopsys ID: 6129927

Severity: B2-Medium

Title: Last Instruction Index Status of Flexible Receive Parser Updated in Incorrect Field of the Receive Descriptor

Impacted Configurations: Configurations where Enable Flexible Receive Parser feature is selected.

Defect Summary:

When all the following conditions are met, the RDES2[26:19] or MADRM field contains the last instruction index of the Flexible Receive Parser(FRP).

- The FRP is enabled (FRPE field of MTL_Operation_Mode register is 1) and
- The last Instruction in RX Status is enabled (ELIRS field of MTL_RXP_Control_Status register is 1) and
- RXPS field (bits [13:11] of RDES2 of Receive Descriptor) is not equal to 5 (L2 Register Filter Fail and Receive Parser processing is bypassed)

When some or all of these conditions are not met, the RDES2[26:19] or MADRM field contains the matched MAC Address register number or Hash Value.

The RDES2[31:27] and RDES2[18:16] provides the L2/L3 registers filters status irrespective of FRP being enabled or not.

However, due to the defect, the last instruction index status of the FRP is provided in RDES2[23:16] instead of expected RDES2[26:19]. The RDES2[18:16] field is overwritten and RDES2[31:27] field is updated with all-zeros, impacting the status of L2/L3 registers filters.

Impacted Use Cases:

The software enables FRP (the FRPE field of the MTL_Operation_Mode register is 1) and the last instruction in RX status (ELIRS field of MTL_RXP_Control_Status register is 1).

Consequence:

The misplaced FRP last instruction index status field and overwriting of L2/L3 Registers filters status fields impacts the dependent software functions.

Workaround:

The software must interpret RDES2[23:16] as Rx Parser last instruction index status, when the following conditions are met.

- FRP is enabled (FRPE field of MTL_Operation_Mode register is 1) and
- Last Instruction in RX Status is enabled (ELIRS field of MTL_RXP_Control_Status register is 1) and
- The RXPS field (bits [13:11]) of RDES2 of Receive Descriptor) is not equal to 5 (L2 Register Filter Fail and Receive Parser processing bypassed)

The L2/L3 Registers filters status is not provided even if the corresponding filters are enabled.

Otherwise, interpret RDES2[31:16] as L2/L3 register filter status.

Method of Reproducing:

- 1) Setup FRP Instruction Table to pass filter with non-zero last instruction index. Also, enable the FRP, by following the programming guidelines provided in the *Synopsys Controller IP Ethernet Quality-of-Service Databook*.
- 2) Program the ELIRS field of MTL_RXP_Control_Status register to 1, to enable providing the last instruction index in the RX Status.
- 3) Generate and send a packet to the MAC receiver, such that it passes the FRP filter.

Observe that the FRP last instruction index is updated in the RDES2[23:16] of Receive Descriptor, instead of the RDES2[26:19] as expected.

2.18.18 Synopsys ID: 3275809

Severity: B1-Low

Title: Incorrect Description of MAC_Address0_High Register in the Databook

Impacted Configurations:

All DWC_ether_qos configurations in which Synchronize CSR MAC Address to Tx/Rx Clock Domain feature is not selected but Enable Power Management feature is selected.

Defect Summary:

When Synchronize CSR MAC Address to Tx/Rx Clock Domain feature is not selected but Enable Power Management feature is selected in the configuration, the DWC_ether_qos Databook contains incorrect description of the MAC_Address0_High register.

The description in Databook will be corrected as follows:

The MAC_Address0_High register holds the upper 16 bits of the 6-byte MAC address of the station. The first DA byte that is received on the (G)MII interface corresponds to the LS byte (Bits [7:0]) of the MAC_Address0_Low register.

For example, if 0x112233445566 is received (0x11 is first byte received) on the (G)MII as the destination address, the MAC Address0 register [47:0] is compared with 0x665544332211. If the MAC address registers are configured to be double-synchronized to the (G)MII clock domains or if Power Management feature is selected, the synchronization is triggered only when Bits[31:24] (in little-endian mode) or Bits[7:0] (in bigendian mode) of the MAC_Address0_Low register are written. So, the MAC_Address0_High register must be written prior to MAC_Address0_Low register.

For proper synchronization updates, the consecutive writes to the MAC_Address0_Low register should be performed after at the least 4 clock cycles of the slowest clock among the source and destination clock domains.

Impacted Use Cases:

Synchronize CSR MAC Address to Tx/Rx Clock Domain feature is not selected but Enable Power Management feature is selected in the configuration and write access is performed to the MAC_Address0_Low register prior to AC_Address0_High register.

Consequence:

When MAC_Address0_Low register is written prior to MAC_Address0_High register, the existing incorrect value in the MAC_Address0_High register is synchronized to the Tx/Rx Clock domain resulting in incorrect destination address filtering (dropping of the received packets).

Recommendation:

Update the application software to correct the sequence of writing to the MAC_Address0_High and MAC_Address0_Low registers.

Workaround: Not Applicable.

[2.18.19 Synopsys ID: 4974260](#)

Severity: B2-Medium

Title: DMA Incorrectly Fetches Invalid Descriptors When Tail Pointer is Updated with Same Address

Impacted Configurations: All non-EQOS-CORE and non-EQOS-MTL Configurations.

Defect Summary:

The DMA fetches and processes descriptors up to the one located prior to the Tail Pointer (DMA_CH(#i)_TxDesc_Tail_Pointer and DMA_CH(#i)_RxDesc_Tail_Pointer). It generates Buffer Unavailable Interrupt, and enters suspend state. When the software updates the Tail Pointer, the DMA exits the suspend state, fetches, and processes further descriptors if the Tail Pointer is moved ahead. Otherwise, the DMA remains in suspend state and generates Buffer Unavailable Interrupt to indicate to the software that there are still no descriptors to fetch and process, even after the Tail Pointer update. This prompts the software to take corrective action.

However due to the defect, when Tail Pointer is updated with same address, the DMA incorrectly exits suspend state, fetches and processes descriptor pointed by Tail Pointer and beyond. This continues if the DMA finds a 1 in the OWN bit position in the descriptors yet to be set or made available to the DMA for processing. This results in unintended packets to be transmitted, impacting performance. Also, processing these packets at the remote end, leads to unintended actions.

To recover, the software must do the following:

When the Buffer Unavailable Interrupt is generated and the DMA Current Descriptor Pointer (DMA_CH(#i)_Current_App_TxDesc and DMA_CH(#i)_Current_App_RxDesc) is equal to or greater than Tail Pointer,

- 1) Software must stop the DMA.
- 2) Setup the descriptors from start of list.
- 3) Program the list address (DMA_CH(#i)_TxDesc_List_Address and DMA_CH(#i)_RxDesc_List_Address).
- 4) Start the DMA.

This ensures that the DMA restarts processing from the first descriptor in the list.

Impacted Use Cases:

The software updates the Tail Pointer with same address, although it is not the recommended procedure.

Consequence:

The DMA processes unintended descriptors and transmits corresponding packets, impacting performance and resulting in unintended actions at the remote end.

Workaround:

The software must not program the Tail Pointer with same value, especially when the Buffer Unavailable Interrupt is due to the DMA hitting the Tail Pointer limit.

Method of Reproducing:

- 1) Generate and send four packets to the transmitter, one descriptor for each packet. Set the descriptor for the fifth packet with OWN bit = 0.
- 2) Program the
 - a). DMA_CH0_TxDesc_Ring_Length to 9.
 - b). DMA_CH0_TxDesc_Tail_Pointer to start address of the fifth descriptor in the list. This indicates that there are 10 descriptors in the list and only first four are owned by DMA.
- 3) Wait for the Transmit DMA Buffer Unavailable Interrupt.
- 4) Read the DMA_CH0_Current_App_TxDesc register. This provides the start address of the fourth descriptor.
- 5) Program the DMA_CH0_TxDesc_Tail_Pointer register, with same address as earlier.
- 6) Wait for the Transmit DMA Buffer Unavailable Interrupt.
- 7) Read the DMA_CH0_Current_App_TxDesc register.

Observe that reading the DMA_CH0_Current_App_TxDesc register provides start address of fifth descriptor, indicating that DMA had exited the suspend state and fetched the fifth descriptor, which is an incorrect behavior.

2.18.20 Synopsys ID: 5121311

Severity: B2-Medium

Title: IEEE 802.1Qav CBS Algorithm Does Not Include Minimum IPG of Packet in Credit Computation

Impacted Configurations: Configurations that include the following:

- Audio Video Bridging Feature as enabled.
- Number of Transmit queues is greater than 1.
- Support for AV in TXQ(#i) is enabled, with i=1 to 7, any one or more values.

Defect Summary:

When Credit Based Shaper (CBS) is enabled for a Transmit Queue (TXQ), the packet that is available in a TXQ is scheduled for transmission when the TXQ accumulates zero or positive credit. When the current credit is negative or when a packet is available in the Transmit Queue and is waiting for the opportunity to be scheduled for transmission, the MAC increments the credit. Equally, during the actual packet transfer, the MAC decrements the credit. The rate at which the MAC increments or decrements the credit depends on the percentage of the total bandwidth that is reserved for the TXQ.

As per the IEEE 802.1Qav standard, the MAC must decrement the credit also when the packet overheads are transmitted. These overheads include the preamble bytes before the start of packet, CRC/FCS bytes, and the minimum 12-byte Inter Packet Gap (IPG) after the end of the packet data transfer.

However, due to this defect, the MAC decrements the credit only up to the last byte of the packet data (that is, the last byte of the Frame Check Sequence (FCS) transfer) and increments the credit during the subsequent nominal IPG period that is associated with that packet.

Impacted Use Cases:

The CBS algorithm is enabled to schedule the transmission of Audio/Video traffic.

Consequence:

As the MAC does not decrement but increment the credit during the trailing IPG period of the packet, the accumulated credit value is higher or larger than expected. As a result, the next packet of that traffic class is scheduled earlier or faster than expected, and the process repeats. Consequently, in each time window for which the bandwidth reservation is measured, the actual bandwidth that the TXQ consumes exceeds the software programmed value.

The percentage of extra bandwidth that the TXQ consumes depends on the packet length. You can estimate it by using the following equation, assuming the nominal 12-byte IPG.

Additional/Extra bandwidth = ((# of packets x 12 bytes) / (Total number of bytes that is transmitted in that window, including the preamble bytes of each packet)) x Fractional bandwidth that is programmed for the TXQ.

For example, if you program a 30% bandwidth for a TXQ which transfers a stream of 100 packets of 128 bytes length, the additional consumed bandwidth is equal to $(30\% \times (100 \times 12) / (100 \times (8 + 128))) = \sim 2.65\%$. Effectively, this TXQ then receives 32.65% of total bandwidth instead of the programmed 30%.

Workaround:

You must compute the additional/extra bandwidth that the TXQ consumes due to the defective algorithm. For this, you must use the abovementioned formula and example, substituting the values with the average length of the packets that are transmitted for the use case. Then, you must program the parameters that determine the fractional bandwidth for this TXQ to reflect the lesser percentage, such that the effective programmed fractional bandwidth is closer to the desired one.

Method of Reproducing:

- 1) Follow the programming guidelines in the *Synopsys Controller IP Ethernet Quality-of-Service Databook* to enable CBS for TXQ1. Program a bandwidth of 30%.
- 2) Continuously generate and send packets of 128-byte size to both TXQ1 and TXQ0, so that they are transmitted back-to-back on the line with the nominal 12-byte IPG.
- 3) Measure the actual bandwidth on the line for packets that are transmitted from TXQ1.

Observe that the measured bandwidth on the line for packets that are transmitted from TXQ1 is 32.65%, instead of the expected 30%.

2.18.21 Synopsys ID: 5584644

Severity: B2-Medium

Title: Extraneous Transmit IPG when EST Feature Enabled

Impacted Configurations: Configurations in which the Enable Enhancements to scheduling Traffic (EST) feature is selected.

Defect Summary:

When the Enhancements to Scheduling Traffic (EST) feature is enabled, the transmit scheduler defers the scheduling of the next packet until the current packet is completely forwarded to the MAC transmitter. This way, any eligible packet that the transmit scheduler receives in a higher priority transmit queue towards the end of the current packet transmission is scheduled over any packet that is already available in a lower priority transmit queue. The transmit scheduler design maintains a counter which triggers when packet forwarding to the MAC transmitter starts and increments on every word that is transferred until the value that is determined based on the packet size (including the preamble and SFD overhead) and operating speed are reached. The value that is determined is the number of words to transmit in the packet, including the overheads (target count) on the line.

However, due to the defect, an extraneous IPG (that is, more than the programmed minimum IPG) is observed even when back-to-back packets are available for scheduling.

In SPRAM configurations, the packet data read from the transmit queue in the application clock domain is written to the asynchronous FIFO. The packet data is read out from asynchronous FIFO in the MAC transmitter clock domain. The counter is incremented when the data word is read out from the asynchronous FIFO.

When the counter reaches the target count, the indication is synchronized back to the application clock domain to enable the scheduling of the next eligible packet. The waiting period until the current packet is completely forwarded to the MAC transmitter for scheduling the next eligible packet, and the crossing of the signals to and from the application clock domain and the MAC transmitter clock domain (CDC delays) result in a delayed scheduling of the next eligible packet which, in turn, reflects in an extraneous IPG.

The IPG is approximately 10 to 12 clock cycles of the slowest of the two clocks (converted to bit times based on the operating speed) based on the frequency and the phase relationship between the application and the MAC transmitter clocks.

In asynchronous two-port SRAM configurations, although there is no CDC delay, the waiting period until the current packet is completely forwarded to the MAC transmitter for scheduling the next eligible packet results in an extraneous IPG of approximately two clock cycles of the MAC transmitter clock, converted to bit times based on the operating speed.

System Usage Scenario:

The software enables the Enhancements to Scheduling Traffic (EST) feature in the MAC to schedule the Time Sensitive Networking (TSN) traffic.

Consequence:

The extraneous IPG impacts the number of packets that can be scheduled in the Time Interval (TI) or slot/row of the EST Gate Control List (GCL). The remote end accepts or drops packets that are received after the expected time based on implementation. The extraneous IPG impacts performance due to the delay in packet transmission.

Workaround:

The software must program the Time Interval (TI) to accommodate the extraneous IPG based on the number of packets that are expected to be scheduled. For example, for SPRAM configurations in one-Gbps speed mode, the extraneous IPG is 10 to 12 clock cycles and the MAC transmitter clock is slower. Therefore, the TI must be increased by at least a value that is equal to (clock period (8 ns) * extra IPG (12)). That is, by 96 ns per data packet.

Method of Reproducing:

- 1) Program the PS and FES fields of the MAC_Configuration register to 0 to enable one-Gbps speed mode.
- 2) Program the EST GCL field.
 - For information, see the description of this field in Chapter 18, "Register Descriptions" in the *Synopsys Controller IP Ethernet Quality-of-Service Databook*.
 - The TI value must exactly fit five packets, including the preamble, SFD overhead, and minimum programmed IPG.
- 3) Start the GCL execution.
 - For information, see "Programming the EST Registers" in Chapter 20, "Programming" in the *Synopsys Controller IP Ethernet Quality-of-Service Databook*.
- 4) Generate and send packets continuously to the MAC transmitter.
- 5) Monitor the GMII transmit for transmitted packets.

Only four packets are scheduled in each row of the GCL field instead of the expected five packets.

2.18.22 Synopsys ID: 5575379

Severity: B2-Medium

Title: Incorrect Execution of EST GCL Iterations When Extended Gate Control Enabled

Impacted Configurations:

Configurations with the Enable Enhancements to Scheduling Traffic (EST) feature selected (DWC_EQOS_AV_EST = 1).

Defect Summary:

When the Extended Gate Control feature is enabled, the software programs the Cycle Time Register (CTR) to a value that exceeds either of the following:

- The sum of Time Intervals (TIs) on the Gate Control List (GCL) rows (that is, the extended gate controls).
- The subsequent reprogramming of the number of GCL rows (LLR) and/or TIs.

During the time interval between the end of the last row of the GCL and the value of the CTR (that is, the extended gate controls time interval), the MAC opens the gate controls for all the Transmit Queues (TxQs). Any eligible packet that is in the transmit queues during this interval is scheduled based on the priority of the TxQs. When the extended gate controls interval ends, the MAC executes the next iteration of the GCL.

However, due to the defect, if the difference between the extended gate controls interval and the time that is required to transmit data packets with an individual total size (including the transmission overheads) of 16384 bytes, is less than nine periods of the PTP clock, the next or the running Base Time Register (BTR) value is additionally updated after the first row of next iteration of the GCL, which is incorrect. As a result, the next iteration of the GCL is incorrectly truncated after the first row, as well as any alternate iteration of the GCL.

System Usage Scenario:

The software programs a CTR value that exceeds the sum of TIs of the GCL rows or the subsequent reprogramming of the number of GCL rows (LLR) and/or TIs. Based on the application traffic profile, this results in a difference between the extended gate controls interval and the time that is required to transmit a 16384 bytes data packet of less than nine periods of the PTP clock.

Consequences:

The next iteration of the GCL is truncated after the first row, which is incorrect. The iteration correctly starts after the time value of the next BTR is reached, but incorrectly truncated after the first row (that is, when the remaining rows of the truncated iteration and the extended gate controls interval are not executed).

As a result, the intended data packets are not scheduled or their scheduling is delayed. If the remote end receiver is configured to drop any packets that are not received during the expected time slot, these packets are dropped and the effect cascades to the subsequent packets.

Also, during the extended gate controls interval, the packets are scheduled with no prior check if they fit into the slot. This causes the packet transmission to overrun into the first row of the next iteration of the GCL, which impacts the performance of any other TxQs that are scheduled in the first row.

Workarounds:

The software must split the extended gate controls interval into the following:

- An additional GCL row, and increase the LLR value by one.
- The remaining extended gate controls interval.

The additional GCL row must be programmed with all the gates open and with a TI that:

- Is at least nine periods of the PTP clock.
- Allows the remaining extended gate controls interval to be less than the time that is required to transmit data packets whose individual total size (including the transmission overheads) is 16384 bytes.

For example, in 1-Gbps speed mode, if the software wants to program the extended gate controls interval to a value of 131074 ns, a value that is greater by 2 ns than the time that is required to transmit data packets with an individual total size of 16384 bytes (that is, 131072 ns). For a PTP clock of 50 MHz, the software can program an additional GCL row of 180 ns (that is, nine periods of the PTP clock). This way, the resulting extended gate controls interval is 130894 ns (131074 ns - 180 ns), which is less than 131072 ns.

Method of Reproducing:

1) Program the following:

- Three GCL rows with individual time values of 131074 ns and with gate open for Transmit Queue 0.
- LLR to 3.
- CTR to 524296 ns (3*131074 ns for the three GCL rows + 131074 ns for the extended gate controls interval).
- BTR based on when to execute the GCL.

For more information, see "20.27 Programming Guidelines for EST " in the *Synopsys Controller IP Ethernet Quality-of-Service Databook*.

2) Enable execution of the GCL.

For information, see "20.27.1 Programming the GCL and GCL Linked Registers" in the *Synopsys Controller IP Ethernet Quality-of-Service Databook*.

3) Continuously generate and send to the MAC transmitter packets of sizes that fit into the slots.

4) Monitor the GMII transmitter for transmitted packets.

No packet transmission occurs for the CTR - 1 GCL row of 131074 ns = 393222 ns time interval during the even-numbered iteration of the GCL.

2.18.23 Synopsys ID: 5938448**Severity:** B2-Medium**Title:** Receive DMA-MTL Interface Halts When Descriptor is not Available for Timestamp Status Write**Impacted Configurations:**

Configurations where Application Interface is DMA (EQOS-DMA/AHB/AXI3/AXI4), the Number of DMA Channels on the Receive Side is greater than one, and Enable IEEE 1588 Timestamp Support features are selected.

Defect Summary:

The receive DMA writes the packet data to the buffers pointed by the normal descriptor and updates the receive packet status in the last normal descriptor for that packet. If the Timestamp is captured for the packet, the TSA bit in the RDES1 of the last normal descriptor is set to 1 indicating that the captured Timestamp status is available in the next descriptor. The captured Timestamp is updated in the next descriptor, which is called the context descriptor.

The receive queue can have packets intended for multiple receive DMA. Therefore, when any receive DMA is stopped or suspended due to unavailability of descriptors, the corresponding packets can be flushed to prevent the head of line (HOL) blocking in the receive queue. The complete or remaining partial packet, along with the status is flushed, only when the RPF bit in the DMA_CH(#i)_Rx_Control register is programmed to 1.

The receive queue read controller provides the receive packet status and Timestamp status, if available, along with the last burst of packet data. That is, no separate request is placed by receive DMA. The Timestamp status is provided irrespective of whether there is a descriptor available for updating it as a context descriptor or not.

However, due to the defect, when the received packet data is forwarded to the application, but descriptor is not available to update the captured Timestamp status. The Timestamp status launched by the MTL receive queue read controller on the DMA-MTL interface is neither accepted by the receive DMA nor flushed. This is because when the receive DMA enters suspend state, it does not issue flush when only the Timestamp status is remaining to be accepted. This results in the DMA-MTL interface being parked with the receive DMA until the descriptors are made available, preventing transfer of packets to any other active receive DMA.

Impacted Use Cases:

The software enables Timestamp capturing for received packets and creates only a limited number of receive descriptors.

Consequence:

The receive DMA-MTL interface is halted, resulting in receive queue overflow and data loss.

Workaround:

The software must create at least one additional descriptor when:

- The receive buffer unavailable interrupt is generated with the AIS and RBU status bits in the DMA_CH(#i)_Status register is set to 1, and
- The updated final descriptor contains the TSA bit in the RDES1 normal descriptor and the LD bit in the RDES3 normal descriptor are set to 1.

Method of Reproducing:

- 1) Create seven receive descriptors in the descriptor memory for receive DMA 1. Similarly, create eight receive descriptors in descriptor memory for receive DMA 0.
- 2) Statically map the receive queue 0 and receive queue 1 to receive DMA 0 and receive DMA 1, respectively.
- 3) Initialize the MAC (with Timestamping function) by following the programming guidelines provided in the Synopsys Controller IP Quality-of-Service Databook, version 5.40a.
- 4) Program the TSENA and TSENALL bits of the MAC_Timestamp_Control register, to enable capturing timestamps for all the received packets.
- 5) Generate and send four packets to the receive queue 1 followed by four packets to the receive queue 0 of the MAC receiver.

Observe that the MAC does not forward four packets received in the receive queue 0 to application. Also, the final descriptor updated for the receive queue 1 contains the TSA bit in the RDES1 normal descriptor and the LD bit in RDES3 normal descriptor are set to 1.

2.18.24 Synopsys ID: 3542857

Severity: B2-Medium

Title: Gate Control List Switching is Incorrect for Intermediate Cycles When CTR is Less Than GCL Execution Time

Impacted Configurations:

DWC_ether_qos configurations in which you select Enable Enhancements to Scheduling Traffic (EST) feature.

Defect Summary:

The EST (Enhancements to Scheduled Traffic) scheduler switches to the next Gate Control List (GCL) after executing the current Gate-Control List (GCL) regardless of the difference between the Cycle Time Register (CTR) value, and sum of the Base Time Register (BTR) and Time Intervals (TI) of the GCL rows whose execution is complete. If the GCL execution takes longer than the cycle time, the GCL is truncated at the cycle time, and the subsequent loop begins at $BTR + N * \text{Cycle Time}$, where N represents the iteration number, an integer.

However due to the defect, in the following situations, the GCL incorrectly updates the internal BTR twice. As a result, the GCL skips the execution of the next GCL loop:

- CTR value is lesser or greater than GCL loop execution time.
- The difference between the CTR and the sum of the BTR and Time Intervals of completely executed GCL rows is less than 8 PTP clock periods expressed in ns.

Impacted Use Cases:

The CTR value that you programmed is not equal to GCL execution time. GCL execution time is as follows:
 $BTR + N * \text{sum of time intervals of valid GCL rows}$.

Consequence:

DWC_ether_qos does not schedule the packet in the expected GCL slot.

Workaround:

You can implement the following workaround in the software:

Program the CTR, BTR, and Time Intervals of the GCL rows such that the difference between the CTR and the sum of the

BTR and time intervals of fully executed GCL rows must be greater than 8 PTP clock periods expressed in ns.
Alternatively, the CTR must be equal to the sum of the BTR and Time Intervals of the fully executed GCL rows.

Revision History

Revision	Date	Changes
0.1	2025/01/07	Preliminary version
0.1.1	2025/02/17	Added an erratum <i>ERR_DMA_001</i>
0.1.2	2025/03/10	Added an erratum <i>ERR_DMA_002</i>
0.1.3	2025/03/20	Added an erratum <i>ERR_RGM_002</i>
0.2	2025/04/13	Added an erratum <i>ERR_RGM_003</i> , and a new erratum for ENET (Synopsys ID: 6129927)
0.2.1	2025/04/22	Added ENET errata (Synopsys IDs: 3630627, 3340878, 3275809, 4974260, 5121311, 5584644, and 5575379)
0.2.2	2025/04/27	- Removed the ENET errata (Synopsys IDs: 3630627 and 3340878) because the impacted configurations are not supported; - Added an erratum <i>ERR_Debug_001</i>
0.3	2025/06/10	- Added errata <i>ERR_Debug_002</i> and <i>ERR_LU_001</i>; - Sorted the errata in alphabetical order in <i>Section 1.2 Summary</i> and <i>Chapter 2 Device Errata</i>; - Added Chinese descriptions for all errata in <i>Chapter 2 Device Errata</i>; - Editorial changes
0.3.1	2025/07/14	Fixed content misplacement and omission in Workaround in <i>Section 2.3 ERR_Debug_001</i> and <i>Section 2.4 ERR_Debug_002</i>
0.4	2026/01/08	- Added two ENET errata (Synopsys ID: 5938448 and 3542857); - Added new errata <i>ERR_SCG_001</i> and <i>ERR_HSADC_001</i>
0.5	2026/04/20	- Removed the erratum <i>ERR_SOC_001</i>; - Added a new erratum <i>ERR_SCG_3715113</i>; - Updated all errata IDs to ensure consistency across different chip variants. See Section 1.4 for the ID mapping table.